Splunk4Ninjas - Dashboard Studio

Minimum versions required:
Splunk Enterprise 10.2
Splunk Cloud 10.1.2507

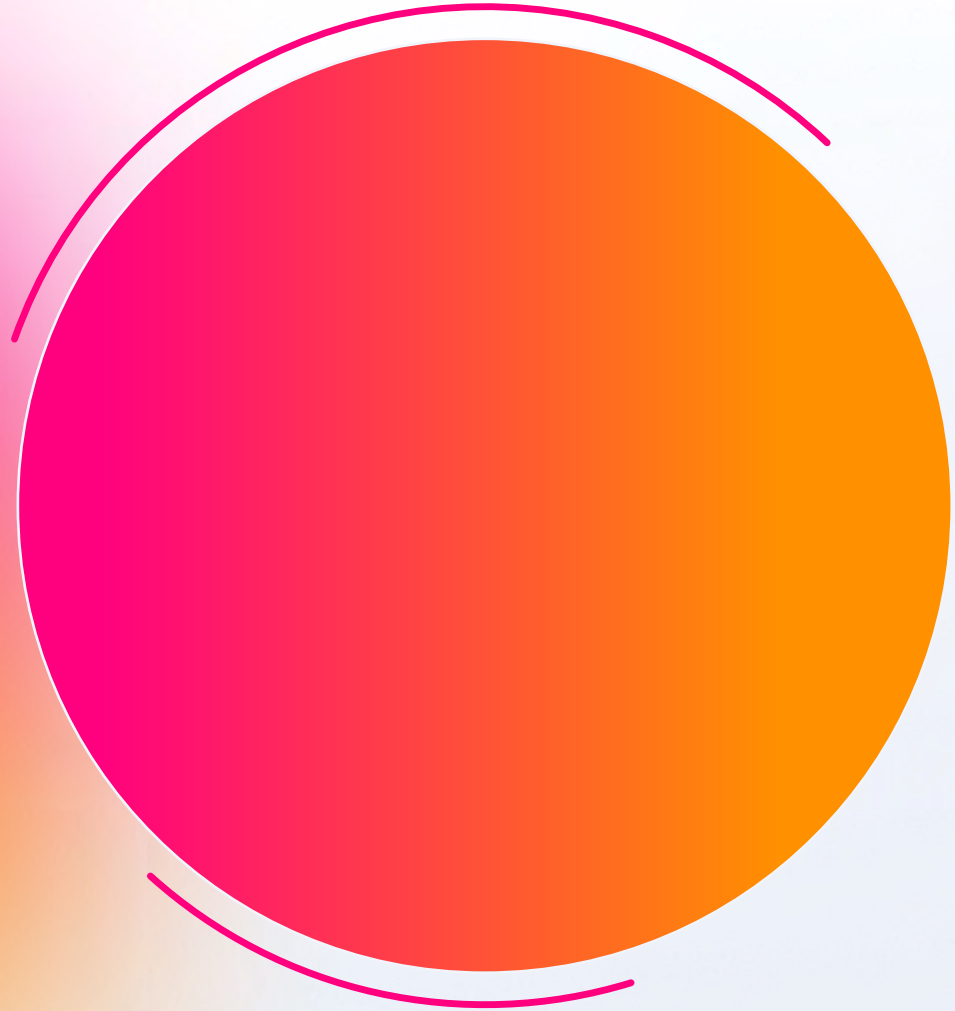


Forward-looking statements

This presentation may contain forward-looking statements regarding future events, plans or the expected financial performance of our company, including our expectations regarding our products, technology, strategy, customers, markets, acquisitions and investments. These statements reflect management's current expectations, estimates and assumptions based on the information currently available to us. These forward-looking statements are not guarantees of future performance and involve significant risks, uncertainties and other factors that may cause our actual results, performance or achievements to be materially different from results, performance or achievements expressed or implied by the forward-looking statements contained in this presentation. For additional information about factors that could cause actual results to differ materially from those described in the forward-looking statements made in this presentation, please refer to Cisco's periodic reports its other filings with the SEC, including the risk factors identified in Cisco's most recent quarterly report on Form 10-Q and annual report on Form 10-K, copies of which may be obtained by visiting the Cisco Investor Relations website at investor.cisco.com or the SEC's website at www.sec.gov. Any projections in this presentation are based on information currently available to Cisco, which is subject to change. The forward-looking statements made in this presentation are made as of the time and date of this presentation. If reviewed after the initial presentation, even if made available by us, on our website or otherwise, it may not contain current or accurate information. We disclaim any obligation to update or revise any forward-looking statement based on new information, future events or otherwise, except as required by applicable law.

In addition, any information about our roadmap outlines or our general product direction is subject to change at any time without notice. It is for informational purposes only and shall not be incorporated into any contract or other commitment. We undertake no obligation either to develop the features or functionalities described, in alpha or beta or in preview (used interchangeably), or to include any such feature or functionality in a future release.

Copyright © 2026 Cisco and/or its affiliates. All rights reserved. Cisco, Splunk and the Cisco and Splunk logos are trademarks or registered trademarks of Cisco and/or its affiliates in the U.S. and other countries. To view a list of Cisco trademarks, go to: www.cisco.com/go/trademarks. Third-party trademarks mentioned in this presentation are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company.



Presenter name

Job title

© 2026 Cisco and/or its affiliates. All rights reserved.

Prerequisites



Prior to beginning this workshop, you should have:

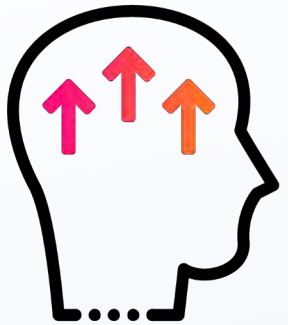
- Taken Splunk4Rookies - Dashboard Studio
- A basic understanding of/built a dashboard in Dashboard Studio

You should already understand how to:

- Navigate the editing UI and have a high-level understanding of the source code
- Add visualizations, searches, and inputs

© 2026 Cisco and/or its affiliates. All rights reserved.

Learning objectives



- **Add interactivity**
  - Configure conditional show/hide
  - Configure token eval
- **Edit source code**
  - Apply options with Dynamic Options Syntax
- **Advanced visualizations**
  - Add multiple data sources to a visualization
  - Add custom dynamic SVGs
- **Apply best practices**
  - Base and chain searches
  - Reusing searches for multiple visualizations
  - Design best practices

© 2026 Cisco and/or its affiliates. All rights reserved.

REGISTRATION 5 MINS

Enroll in today's workshop

Tasks

1. Get a splunk.com account if you don't have one yet:
   https://splk.it/SignUp
2. Enroll in the Splunk Show workshop event:
   https://splunk.show/<uniqueID>
3. Download the hands-on lab guide:
   https://splk.it/S4NDS-Lab-Guide

   Contains step-by-step instructions for all of today's exercises!
4. Download a copy of today's slide deck:
   https://splk.it/S4NDS-Attendee

Goal



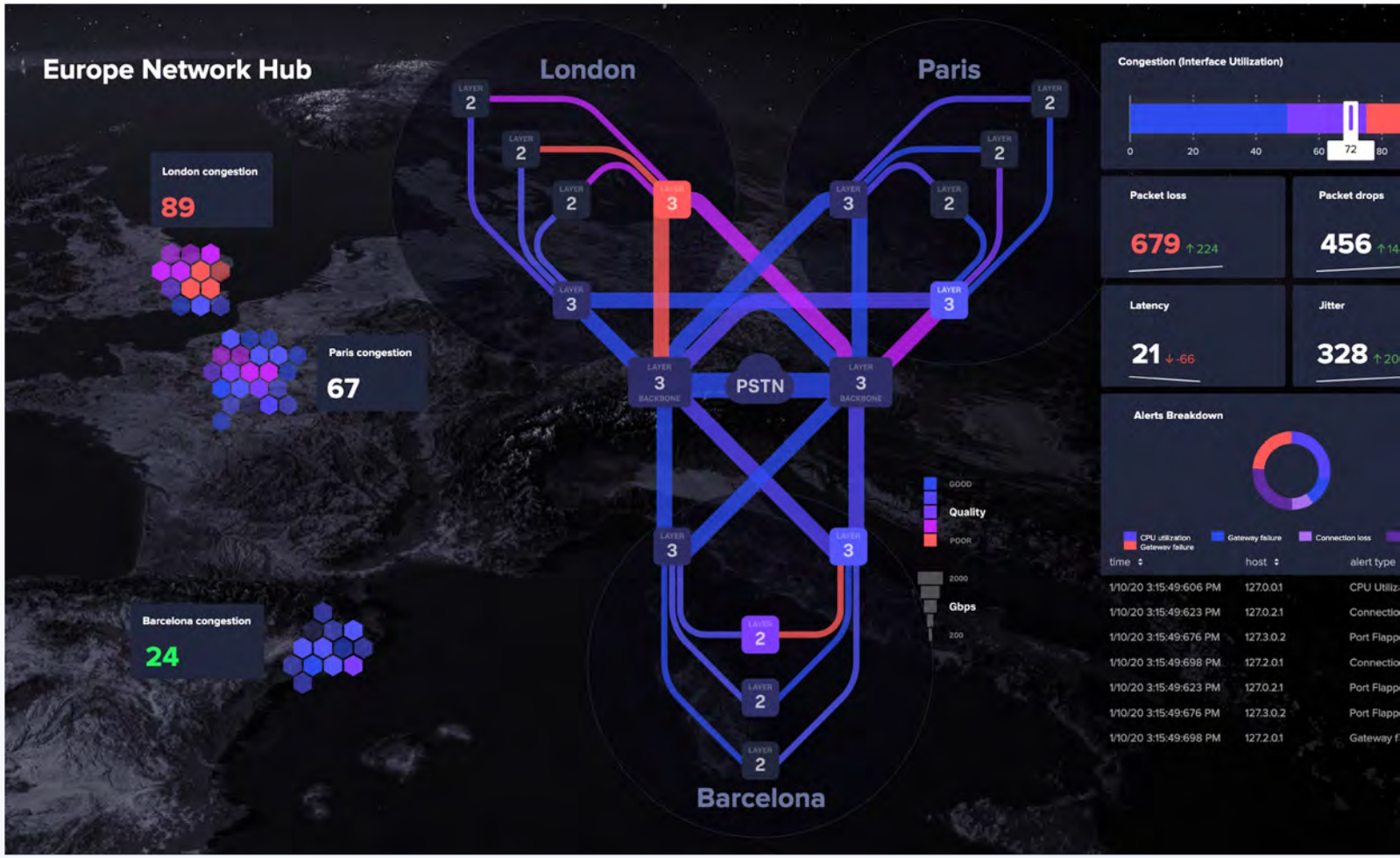
Enroll in today's event

© 2026 Cisco and/or its affiliates. All rights reserved.

About Dashboard Studio

Dashboard Studio is the future of Splunk dashboards

Designed for intuitive point-and-click building, while maintaining flexibility for advanced use cases



© 2026 Cisco and/or its affiliates. All rights reserved.

The Evolution of Dashboard Studio



Splunk Enterprise 9.0 and 9.1

Interactivity improvements

- Set and pass tokens through dashboard drilldowns
- Set tokens from search results
- Show/hide panels



Splunk Enterprise 9.2 and 9.3

Usability and sharing improvements

- Scheduled email export
- UI for all axes chart configuration options
- Bigger code editor
- Classic to Studio conversion



Splunk Enterprise 9.4, 10.0 and 10.2

Customization and data display

- Publish dashboards to allow viewing without logging in
- Custom visualizations support
- Advanced conditional visibility
- Optimized rendering and loading

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

Most features are available in Dashboard Studio

Key features	Classic Dashboards	Dashboard Studio
Data sources ad-hoc, base and post-process, saved searches	✅	✅ UI to easily manage and reuse
Standard charts axes charts, maps, single values	✅	✅
3rd party visualizations	✅	✅ Custom viz support added in 10.2
Flexible layouts	❌ Rigid, requires dev skills to customize	✅ Native support for customization
Inputs	✅	✅
Interactivity link to other pages, set and pass tokens	✅	✅
Tokenization eval, set, condition; search-based tokens	✅	✅
Sharing scheduled email export, export to CSV, on-demand PDF/PNG export	🟡 Limited CSV export, PDF breaks layout	✅
Extensions custom CSS, HTML, JS	✅	❌ some styling natively supported

© 2026 Cisco and/or its affiliates. All rights reserved.

When to use Dashboard Studio vs Classic

Dashboard Studio

- Executive- or customer-facing dashboards
- Visualizations that make use of additional physical objects or spacing
- New or less technical users
- Use cases that don’t require advanced interactivity

Classic Dashboards

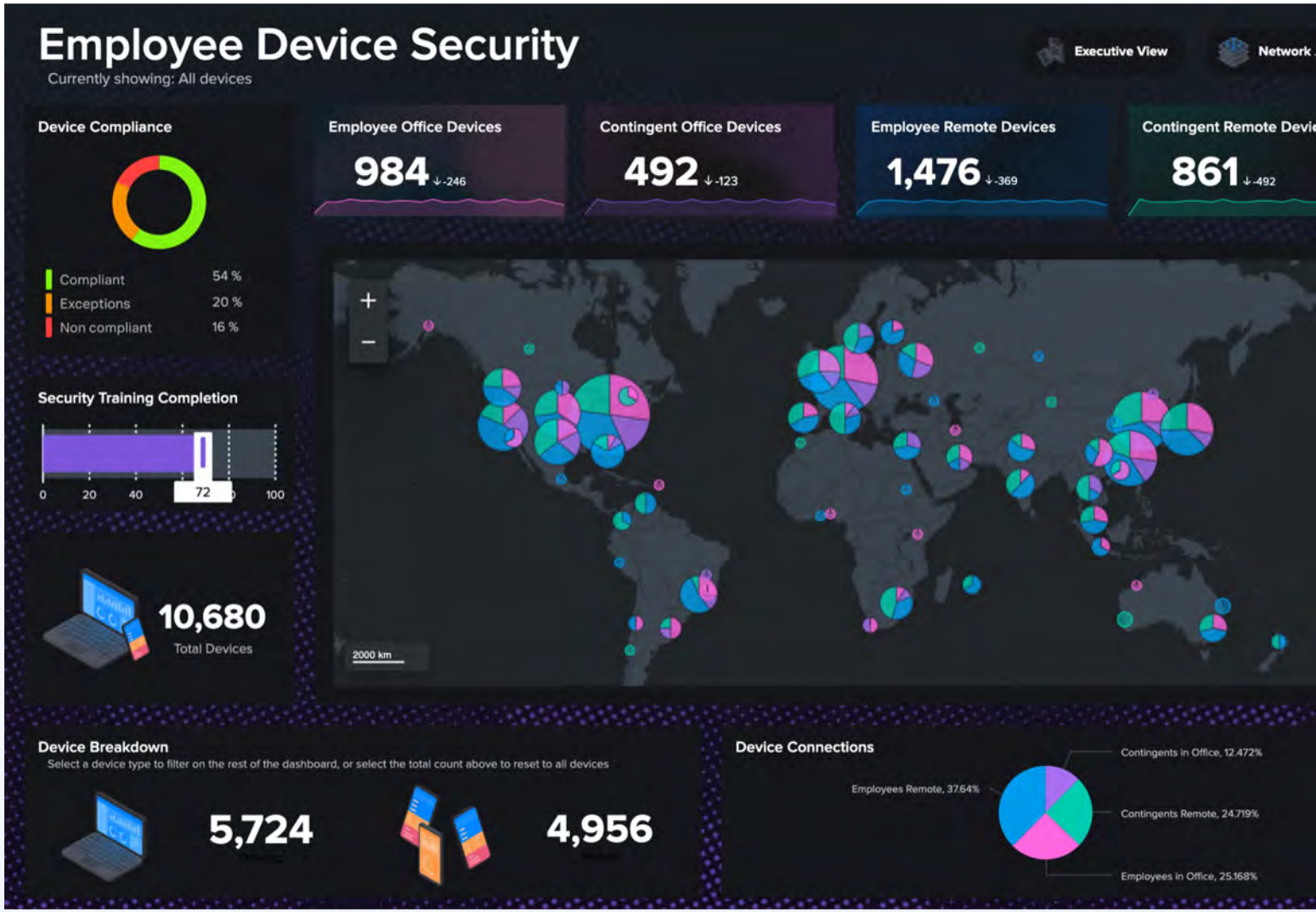
- Dashboards require advanced interactivity or advanced token manipulation
- You need to incorporate complex interactions with custom JS or CSS elements
- You need custom interactions using custom JS.
- You need custom styling

© 2026 Cisco and/or its affiliates. All rights reserved.

High-visibility dashboards

Create intuitive visually appealing dashboards that executives and business partners can understand.

- Enable teams outside of IT and security to leverage Splunk.
- Simultaneously increase data density and readability, especially for display walls.



© 2026 Cisco and/or its affiliates. All rights reserved.

Empower more users

Free up your developer resources with Dashboard Studio.

- No custom development and less app management for customCSS or customJS
- Enable less technical users to self-serve in customizing dashboards

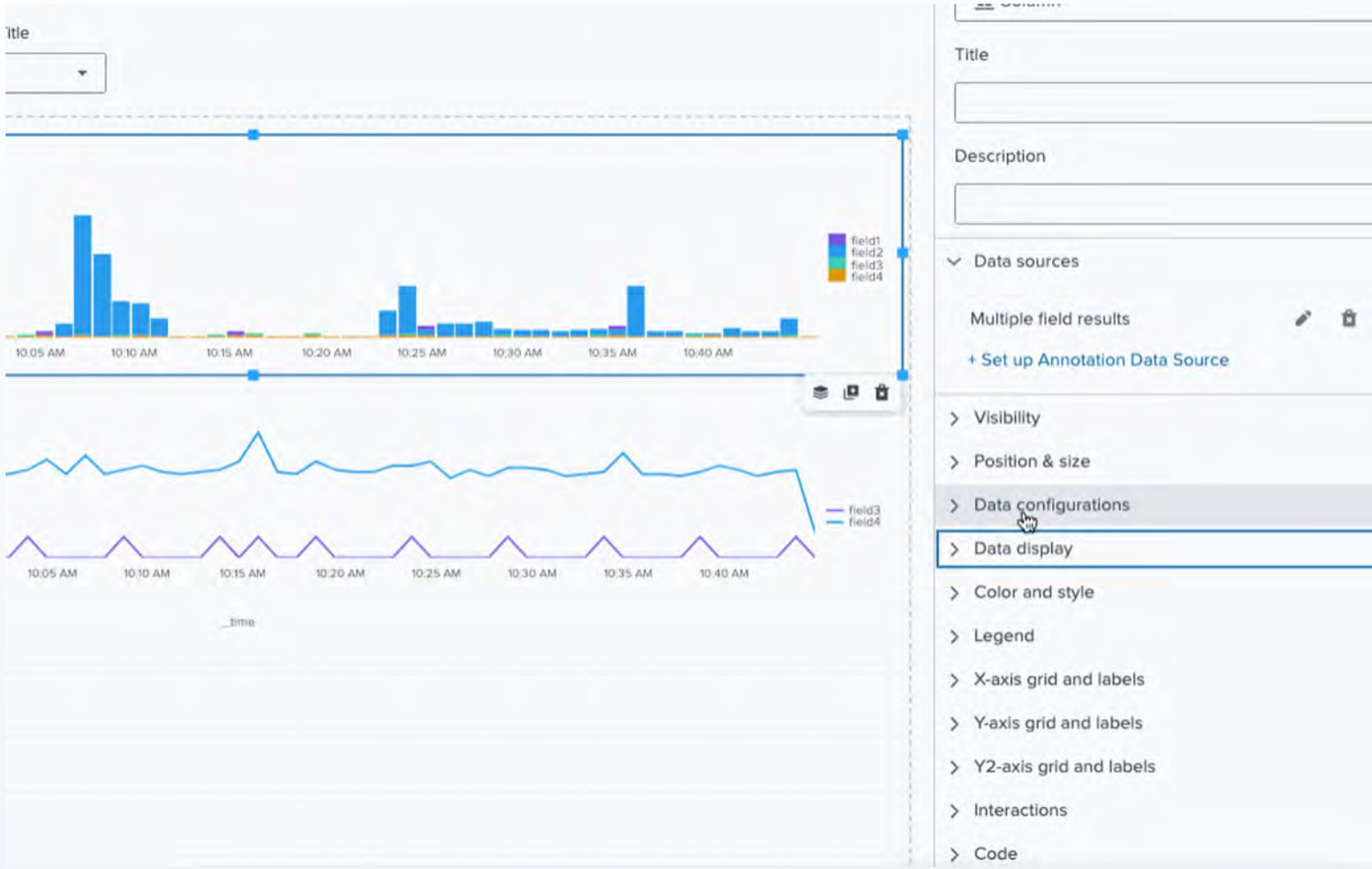


© 2026 Cisco and/or its affiliates. All rights reserved.

Minimize resource consumption

Reuse searches to reduce resource consumption and improve performance.

- Point-and-click configuration to use 1 search for multiple visualizations
  - Intuitive UI to create base and chain searches



Check out the Improving Dashboard Performance and Resource Usage tech talk to learn more.

© 2026 Cisco and/or its affiliates. All rights reserved.

Dashboard Studio unlocks new possibilities

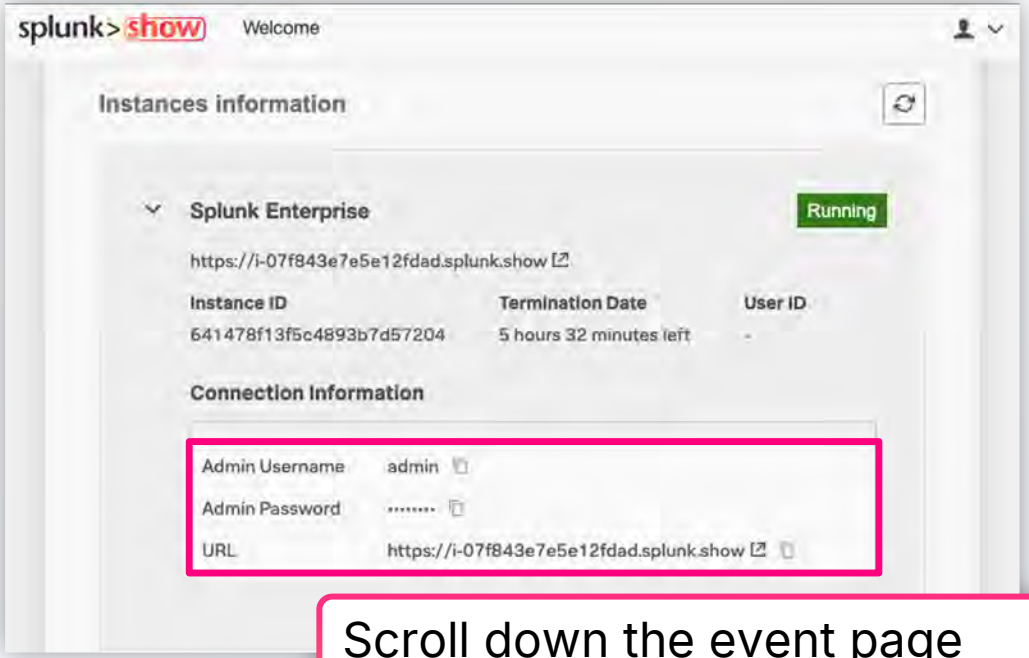
Key use cases	Classic Dashboards	Dashboard Studio
High-visibility dashboards for executives and business partners	❌ Requires custom development	✅ Native support, point-and-click UI
Easy dashboard customization enable end users to apply advanced configurations	❌ Requires custom development	✅ Native support, point-and-click UI
Optimized for performance reduce resource consumption	❌ Limited, requires source code editing	✅ Default behavior, point-and-click UI
Effective visual communication flexible layouts and graphical elements	❌ Rigid layouts, charts only	✅ Flexible layouts, graphical elements
New features and enhancements with each major Splunk release	❌	✅

© 2026 Cisco and/or its affiliates. All rights reserved.

Workshop time!

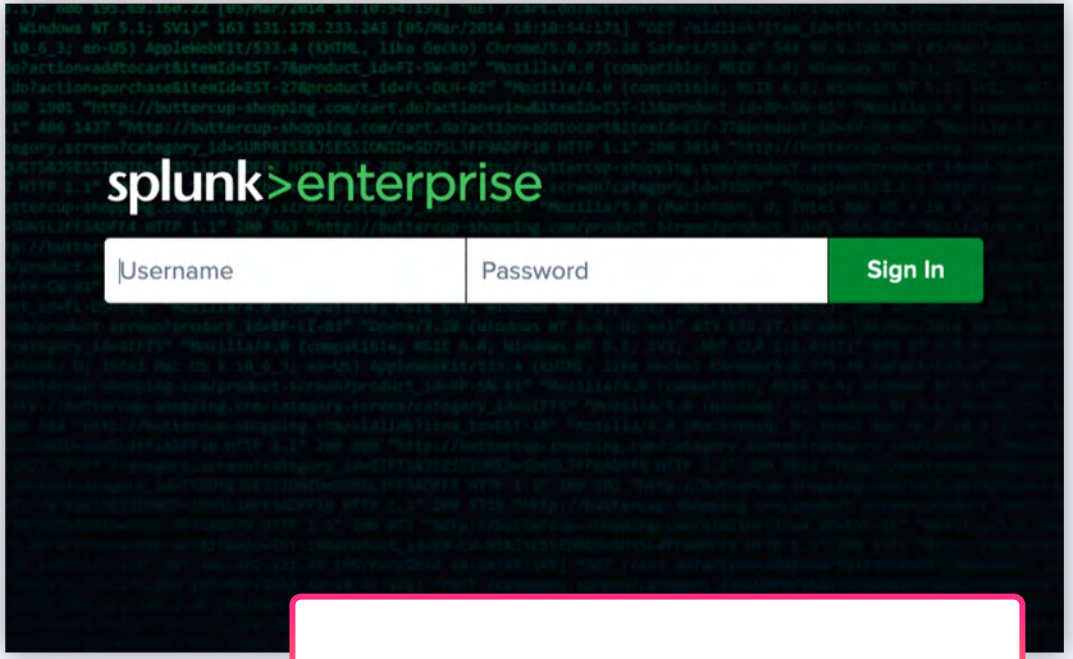
Login to Splunk

Locate your instance URL and credentials in the Splunk Show event https://show.splunk.com



Scroll down the event page and expand the **Splunk Enterprise** section to view your login details

Log in to your Splunk instance



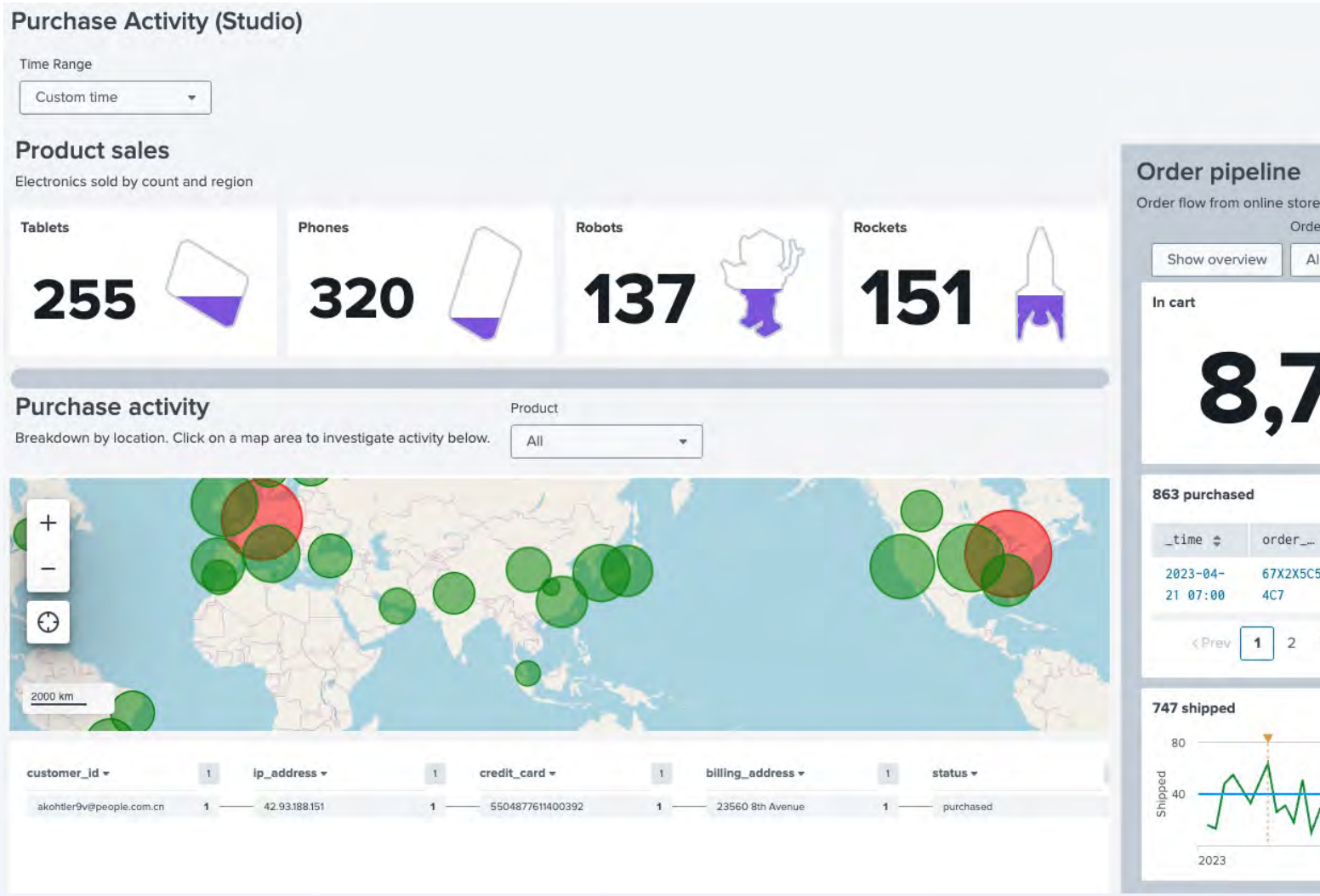
Login using the credentials from Splunk Show

© 2026 Cisco and/or its affiliates. All rights reserved.

Scenario

You are an analyst at Buttercup Games.

You have been tasked with building a dashboard for analyzing customer purchase activity.



© 2026 Cisco and/or its affiliates. All rights reserved.

Dashboard use case and requirements

Use case

Your dashboard will provide insights into product purchase trends such as popular products and order pipeline details for Buttercup Games managers and analysts.

The managers only need a high-level overview, but analysts need to toggle between different views to drill into specific order information and investigate suspicious credit card activity.



Requirements

- Interactive inputs that allow users to toggle between different views for order pipeline status
- Insights into product purchasing trends and metrics
- Insights into abnormal credit card behavior

© 2026 Cisco and/or its affiliates. All rights reserved.

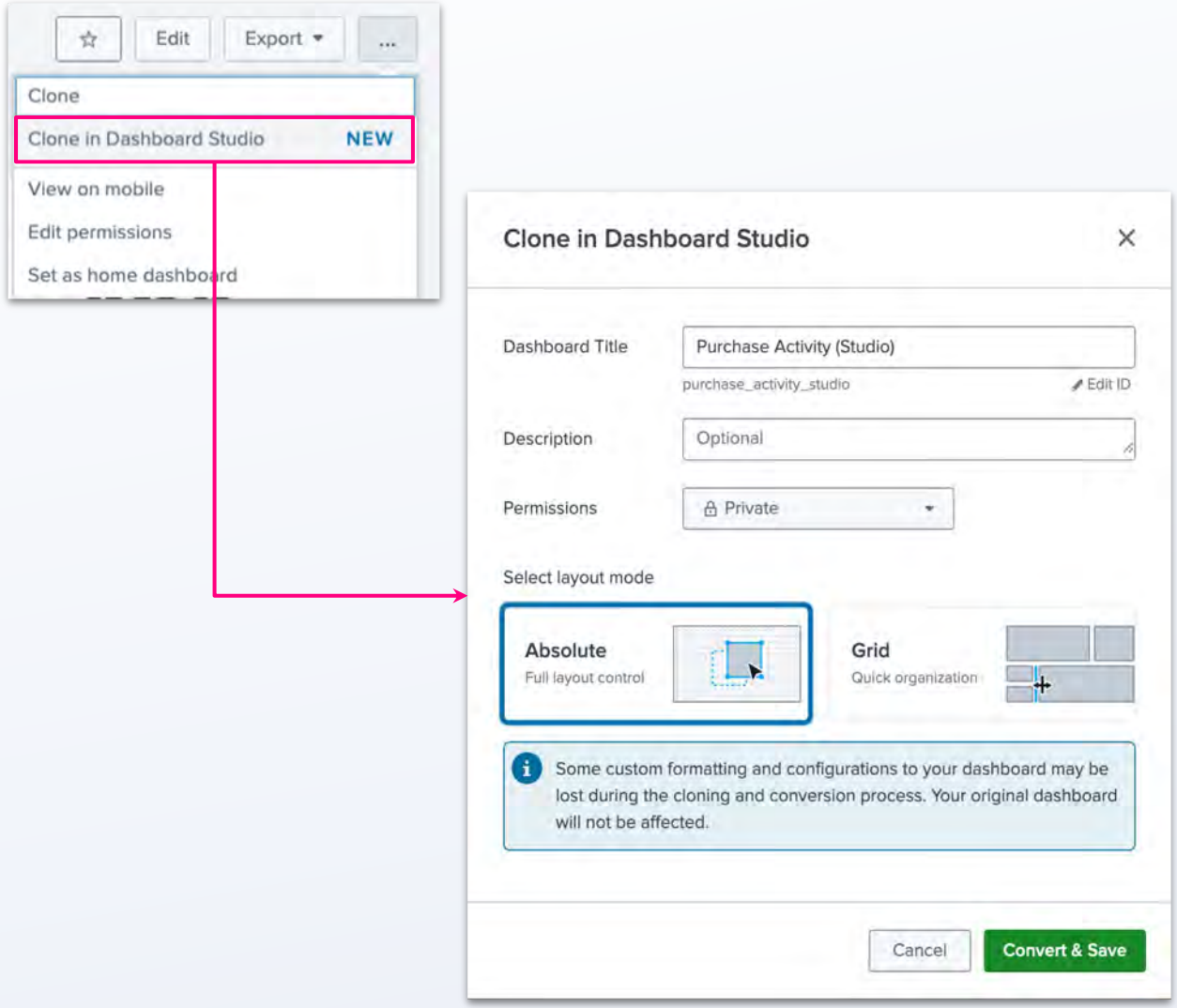
Exercise 1:
Set up your environment

Convert a Classic dashboard

Instead of building a net-new dashboard, let's **clone** an existing Classic dashboard!

1. Open the **Purchase Activity (Classic)** dashboard.
2. Expand the **...** menu and select **Clone in Dashboard Studio**.
3. Update the **Dashboard Title** to **Purchase Activity (Studio).**
4. Select **Absolute** layout.
5. Select **Convert & Save.**

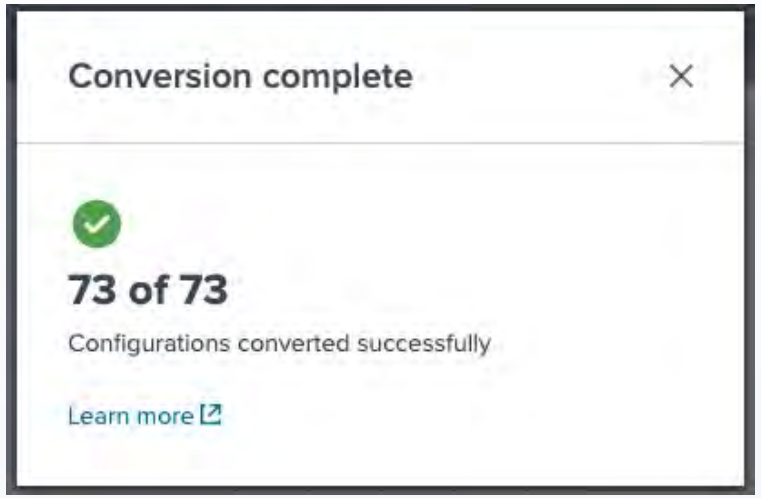
Note: The converted canvas size should be sufficient, but feel free to make the canvas larger as needed to more easily fit your visualizations.



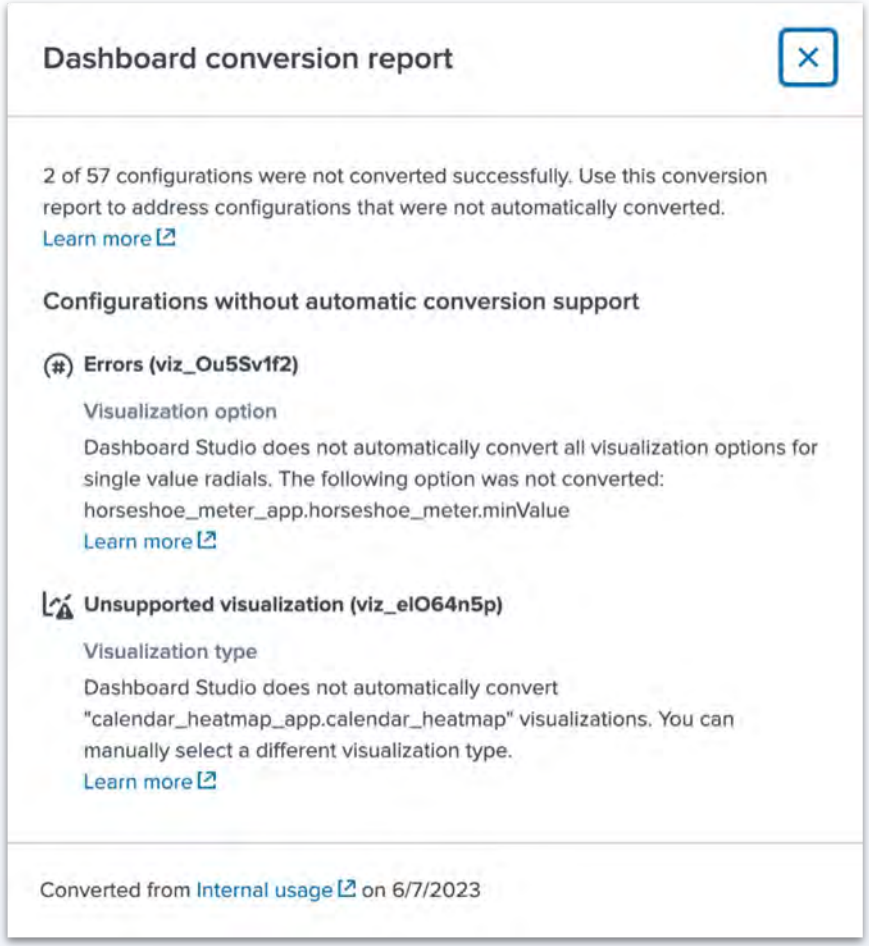
© 2026 Cisco and/or its affiliates. All rights reserved.

Conversion report

Upon conversion you probably saw a similar popup:



In this case, all conversions were converted, but if there were configurations that were not converted, you are prompted to view a detailed report that provides more information about what exactly did not convert.



Example conversion report

© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 2: Configure product purchases single value visualizations

Resize product single value visualizations

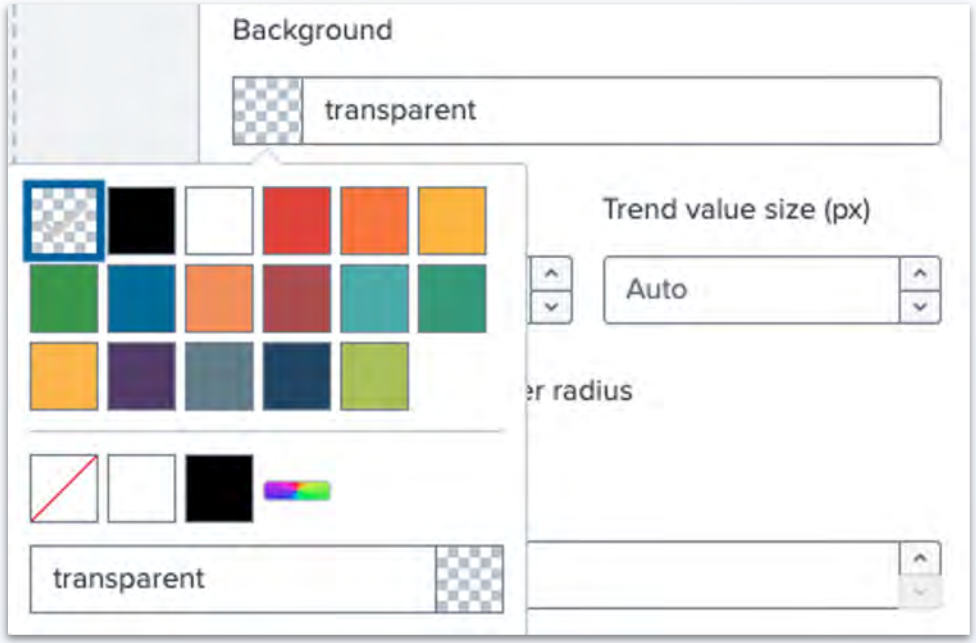
1. Multiselect the single value visualizations:
   - If using macOS: CMD + click.
   - If using macOS or Windows: click and drag to highlight.
2. Bulk update the size of the visualizations to be **140 x 140**.



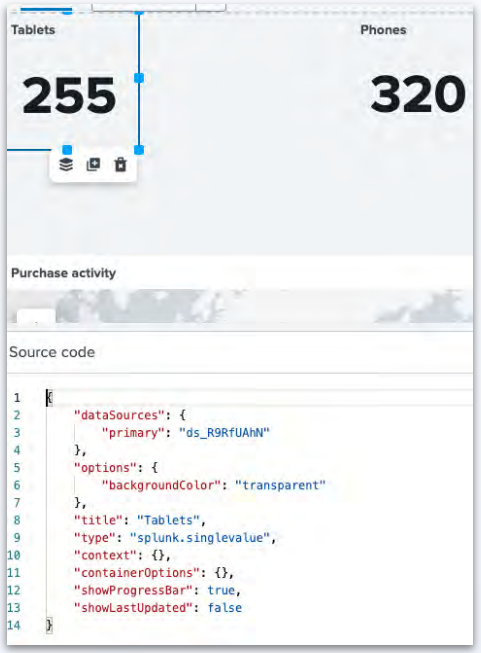
© 2026 Cisco and/or its affiliates. All rights reserved.

Set single value visualizations' background color to transparent

Use the UI under Color and style to set Background color to transparent:



Copy/paste source code between objects:

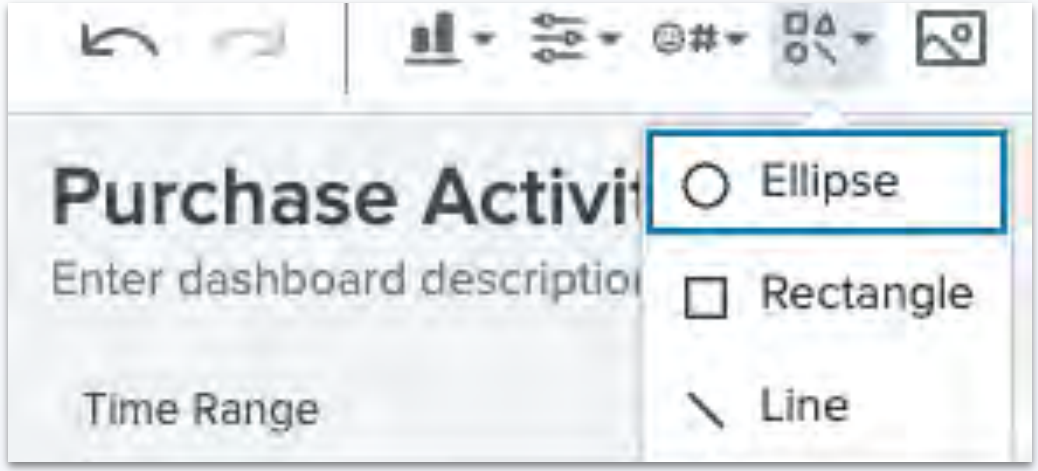


© 2026 Cisco and/or its affiliates. All rights reserved.

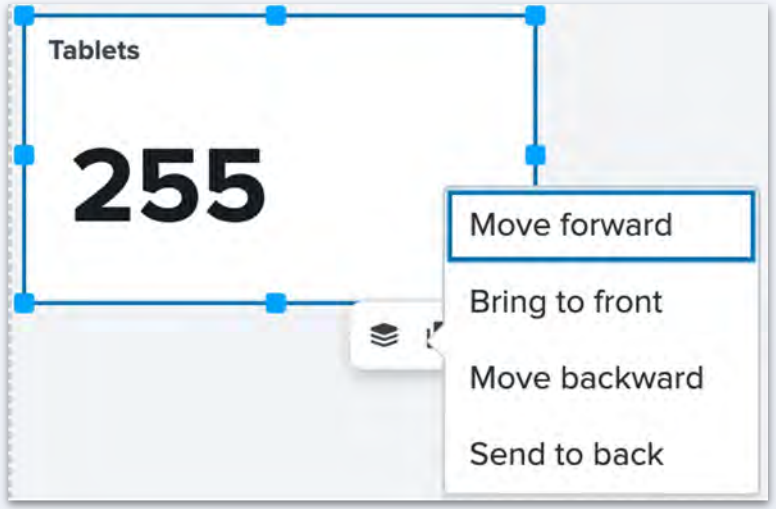
Rearrange the single value visualizations

3. Add a rectangle:
   - Width: 250
   - Height: 140
   - Fill: white
   - Stroke: transparent
4. Position the rectangle behind a single value visualization, using the layering options.
5. Position the single value visualization to be on the left, saving room for an SVG on the right later.
6. Clone the rectangle to repeat the steps for the other KPIs.

1



2



© 2026 Cisco and/or its affiliates. All rights reserved.

Reposition the single value visualizations

8. Reposition the single value visualizations so that:

   - There is room above them for a custom label and description.
   - They take up the left two thirds width of the canvas.
   - There is a little bit of separation between them and the Purchase activity map.

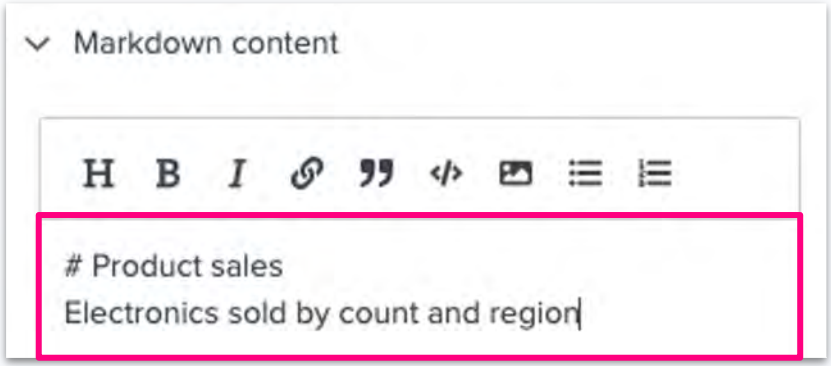


© 2026 Cisco and/or its affiliates. All rights reserved.

Add custom label and divider

9. Add a Markdown text component and position it above the single value visualizations:

   # Product sales
   Electronics sold by count and region



10. Add a rounded rectangle to serve as a divider between the single value visualizations and map:

    a. Set the stroke color to transparent.

    b. Set radius to 10.

11. Position the rectangle between the single value visualizations and the map. The width should match the width of the 4 single value KPIs.



© 2026 Cisco and/or its affiliates. All rights reserved.

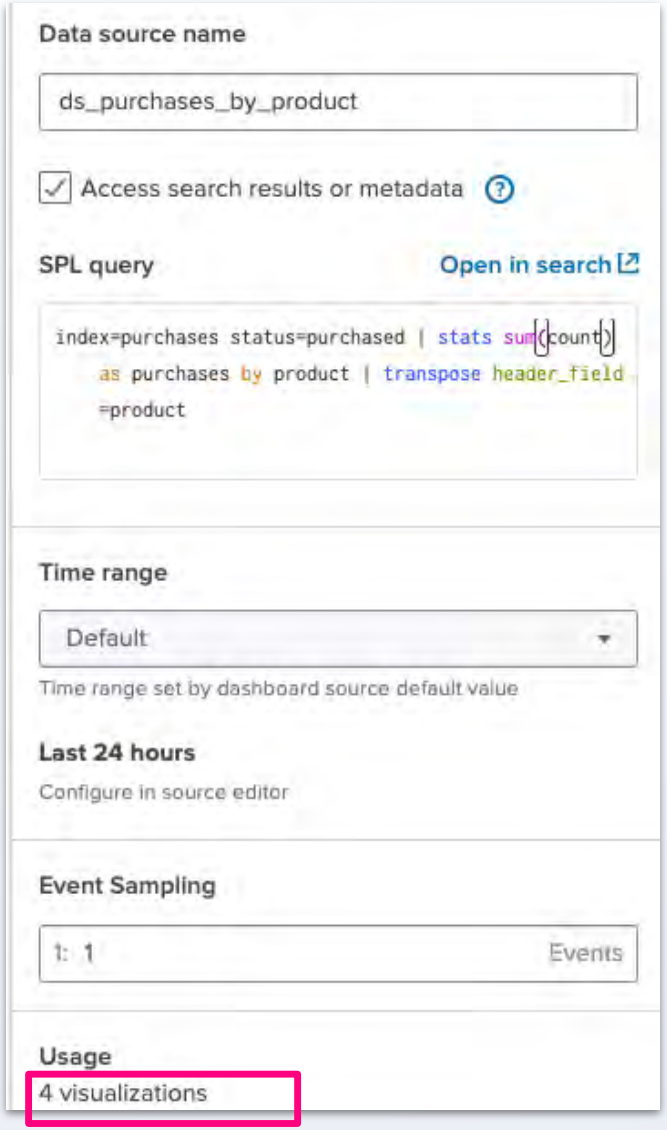
Update the single value visualizations to use a single search

Dashboard Studio allows you to select the field you want to display from a given search. This allows us to consolidate the number of searches needed in the dashboard. Now we'll update each product search to use **ds_purchases_by_product**.

12. Select each product KPI and remove its data source by clicking the crossed-out pencil icon.

13. Select **+ Set up primary data source** and select the **ds_purchases_by_product** data source.

14. Under **Data configurations**, select the appropriate field for the single value from the **Value** dropdown.

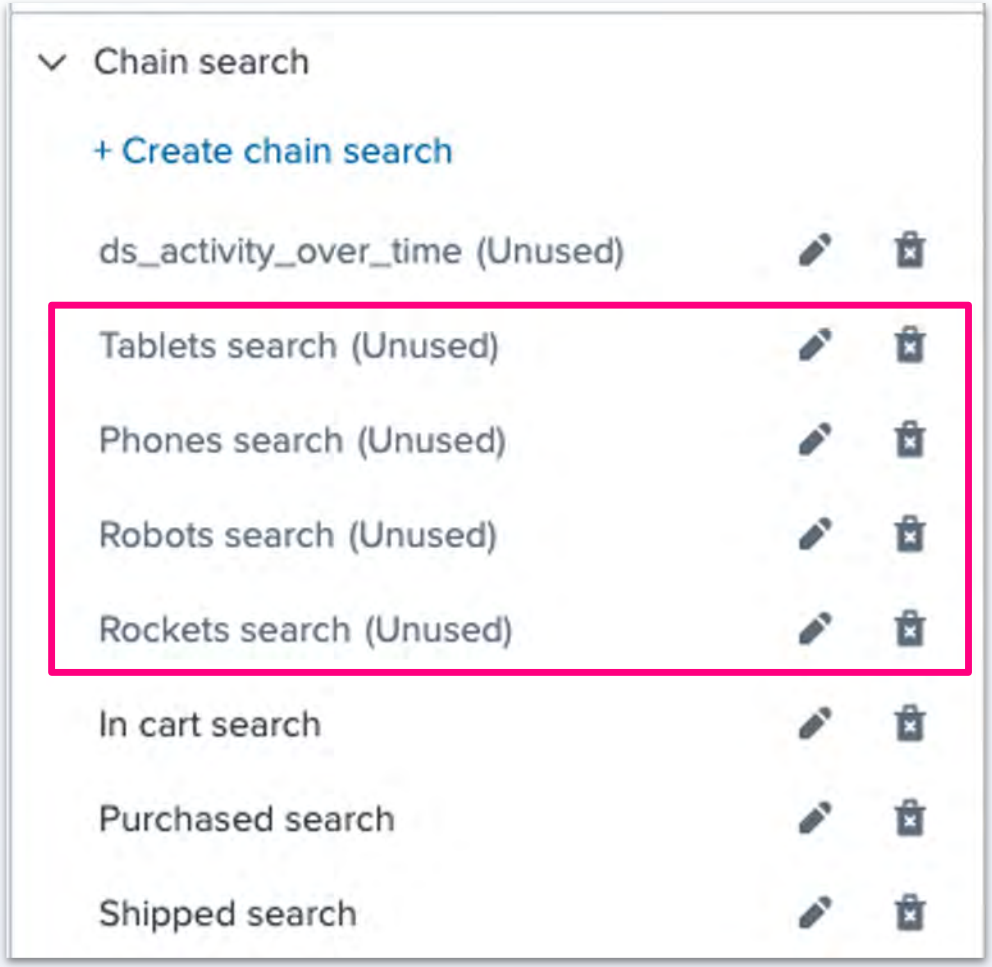


© 2026 Cisco and/or its affiliates. All rights reserved.

Delete the unused chain searches

Now that we don't need the individual chain searches for each product type, we can delete them.

15. Navigate to the **Data source overview** panel by selecting the database icon in the toolbar.
16. Under **Chain search**, delete the unused searches.



Checkpoint! Save your dashboard periodically so you don't lose your changes.

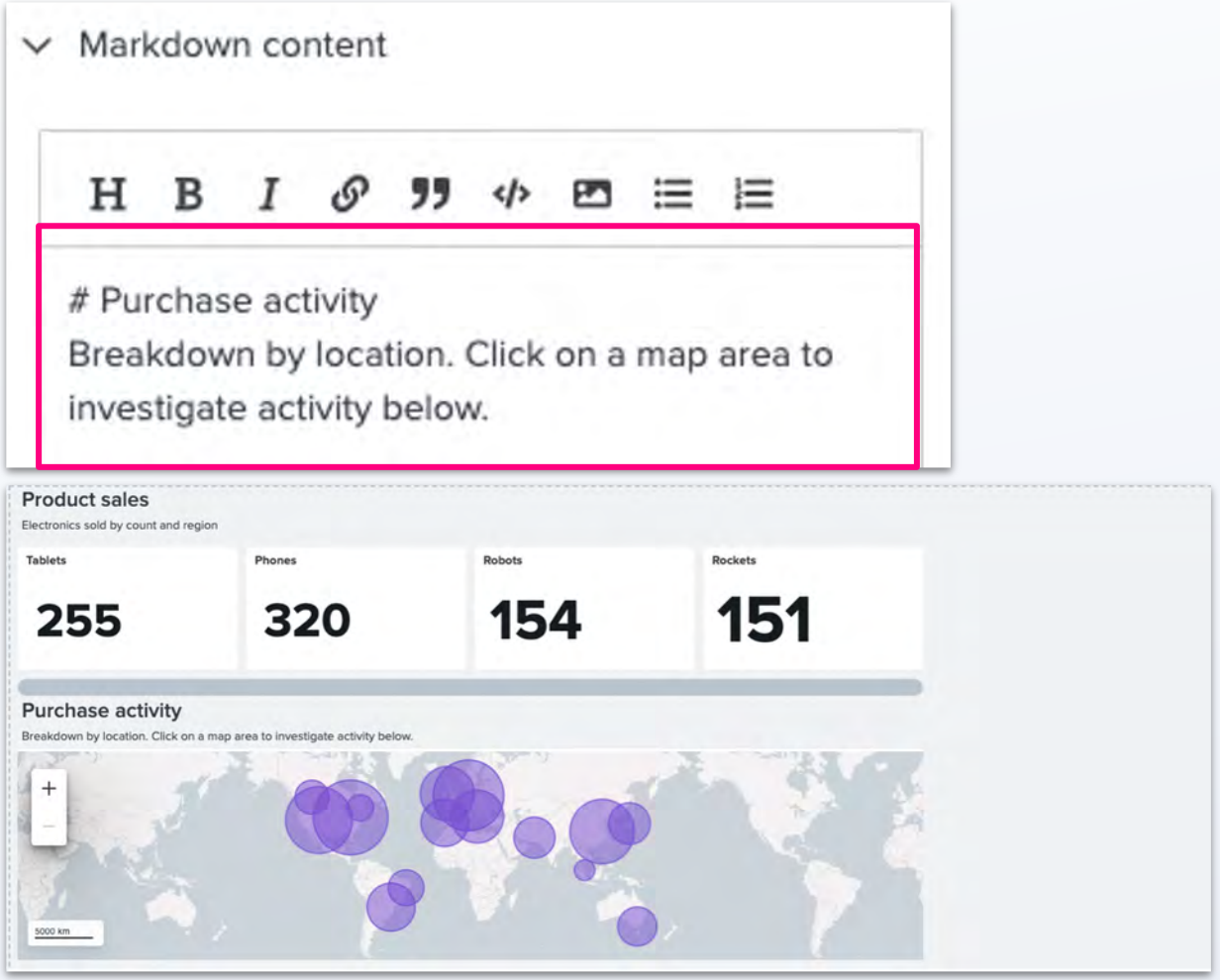
© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 3: Customize the map and add a link graph

Resize and reposition the map

1. Resize the map width to match the divider and change its height (~300 px).
2. Move it down to make room for a custom label.
3. Use Markdown to add a label:
   # Purchase activity
   Breakdown by location. Click on a map area to investigate activity below.

   You may need to move the other charts off the canvas to make room.
4. Remove the map visualization title.
5. Under **Data configurations**, under **Bubble size**, deselect all fields except for **products_purchased (number).**

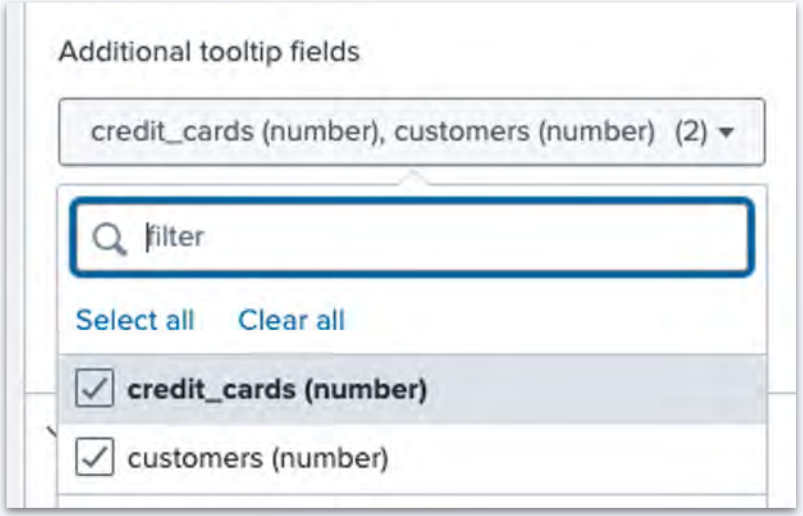


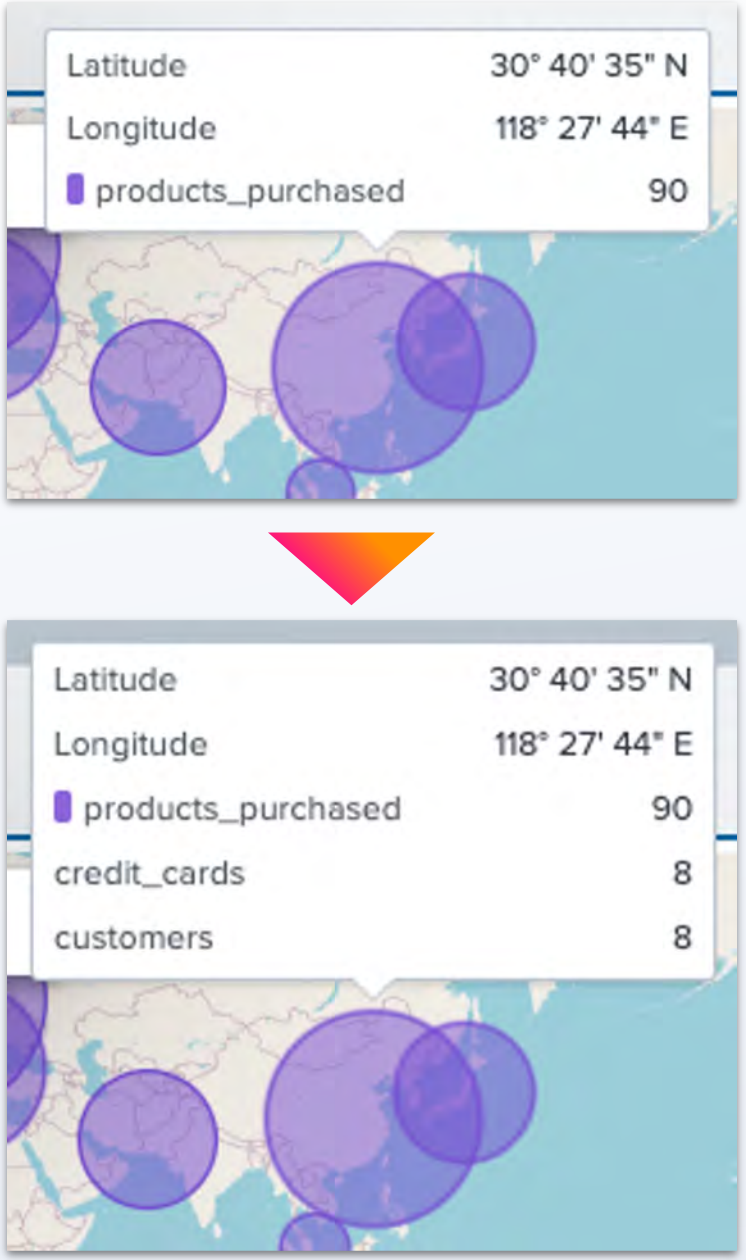
© 2026 Cisco and/or its affiliates. All rights reserved.

Add additional tooltip fields

You can specify additional field values to display in your map tooltips.

6. Under **Data configurations,** under **Additional tooltip fields,** select:
   - credit_cards (number)
   - customers (number)





© 2026 Cisco and/or its affiliates. All rights reserved.

Add a custom tile server

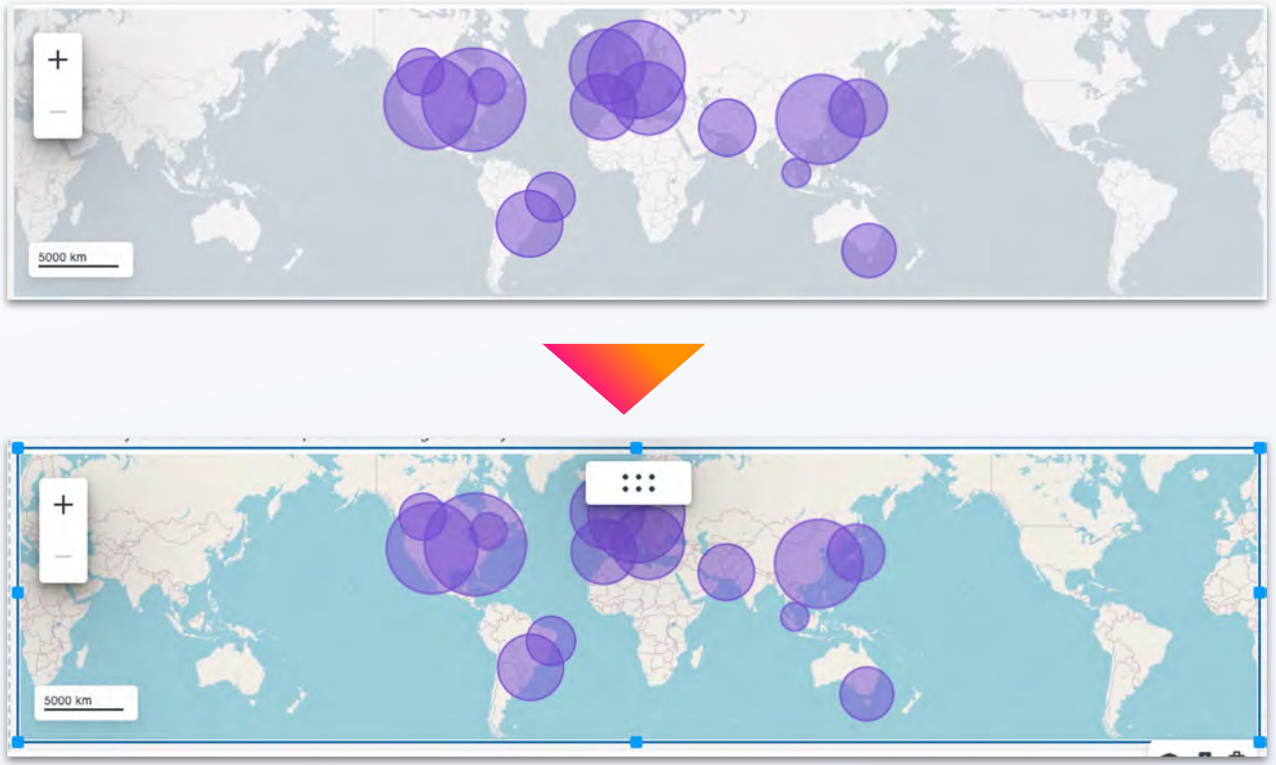
About custom tile servers

- You must specify the tile server type (e.g. vector or raster).
- You can find map tile servers online.
  - Examples: OpenStreetMap, OpenMapTiles, MapTiler, etc.

7. Under **Color and style**, do the following:

   a. Set Base layer tile server: **https://tile.openstreetmap.org/{z}/{x}/{y}.png**

   b. Set Base layer tile server type to Raster.



© 2026 Cisco and/or its affiliates. All rights reserved.

Set tokens on click on Map

8. On the map, under Interactions, click + Add interaction and on the dropdown select Set tokens.
9. Specify the following, with no default values:

   a. Token name: mapBoundsEast
      Token value: row._geo_bounds_east.value

   b. Token name: mapBoundsWest
      Token value: row._geo_bounds_west.value

   c. Token name: mapBoundsSouth
      Token value: row._geo_bounds_south.value

   d. Token name: mapBoundsNorth
      Token value: row._geo_bounds_north.value

Note: You can configure these drilldowns in the UI or write it into the source code directly. Copy pasting from the lab guide might lead to errors with line breaks.

```
"eventHandlers": [
    {
        "type": "drilldown.setToken",
        "options": {
            "tokens": [
                {
                    "token": "mapBoundsNorth",
                    "key": "row._geo_bounds_north.value"
                },
                {
                    "token": "mapBoundsEast",
                    "key": "row._geo_bounds_east.value"
                },
                {
                    "token": "mapBoundsSouth",
                    "key": "row._geo_bounds_south.value"
                },
                {
                    "token": "mapBoundsWest",
                    "key": "row._geo_bounds_west.value"
                }
            ]
        }
    }
],
```

© 2026 Cisco and/or its affiliates. All rights reserved.

Add a link graph

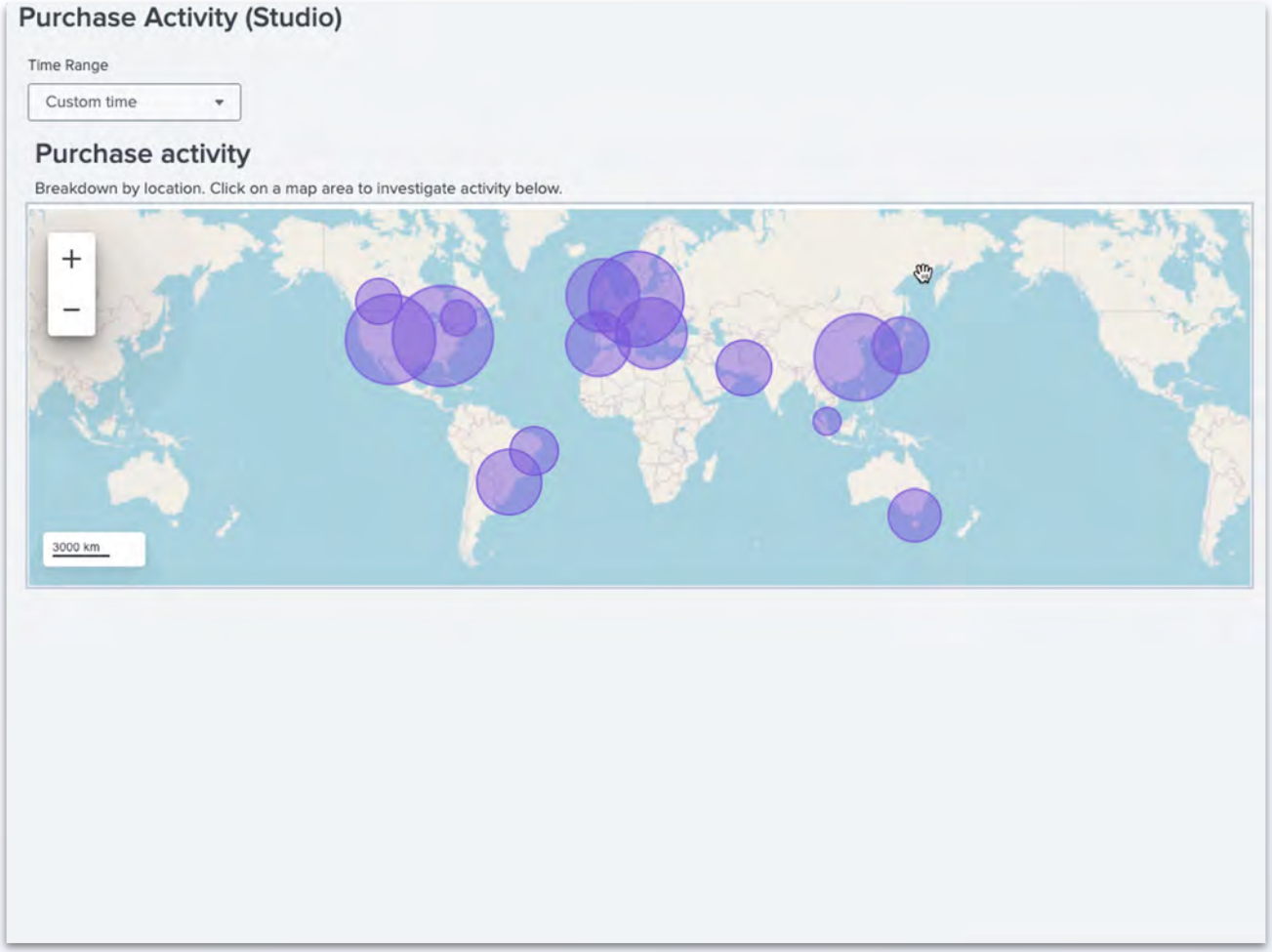
10. Add a link graph under your map.

11. Create a new search titled **"Activity in a highlighted area"**:

```
index=purchases status=purchased
| iplocation ip_address
| where (lat>=$mapBoundsSouth$ and lat<=$mapBoundsNorth$)
  and (lon>=$mapBoundsWest$ and lon<=$mapBoundsEast$)
| table customer_id ip_address credit_card
  billing_address status
```

12. Under **Visibility**, check the **When data is unavailable, hide element** box.

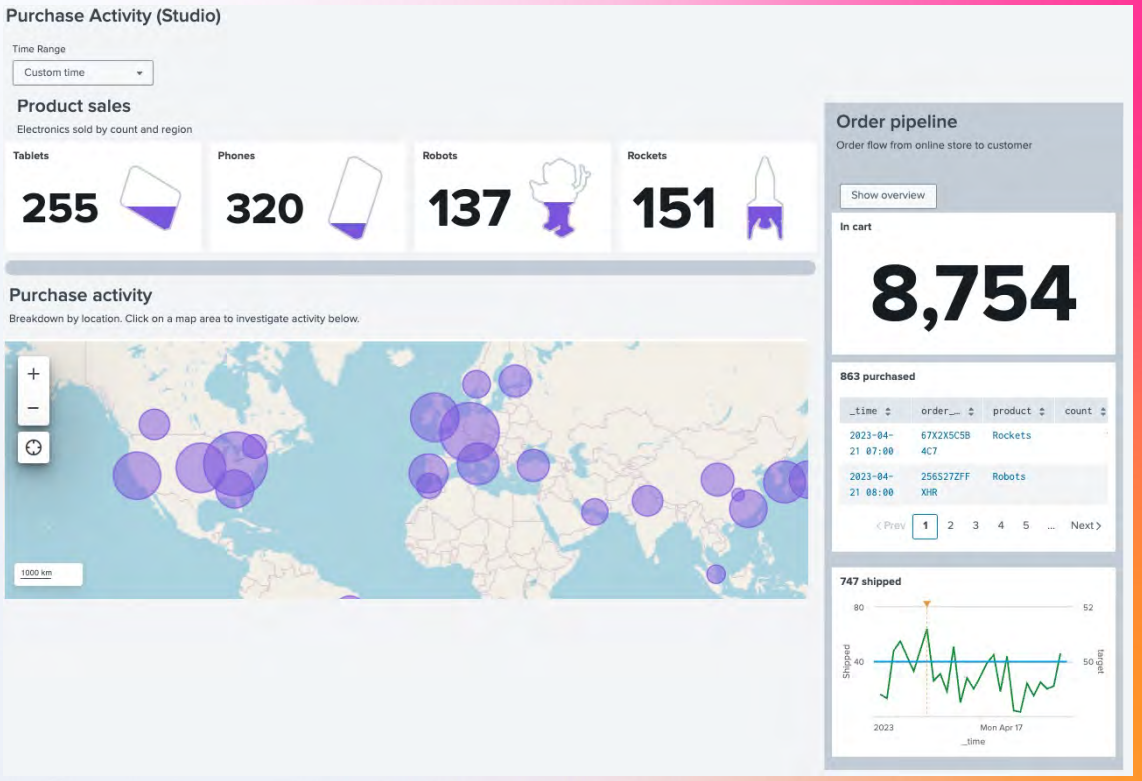
13. Save, and try it out in View mode!



Hint: Select a map bubble to set tokens and observe the link graph's appearance.

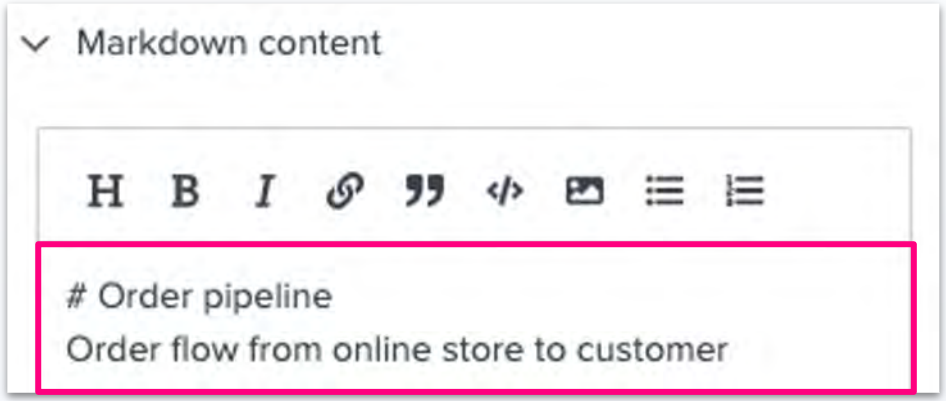
© 2026 Cisco and/or its affiliates. All rights reserved.

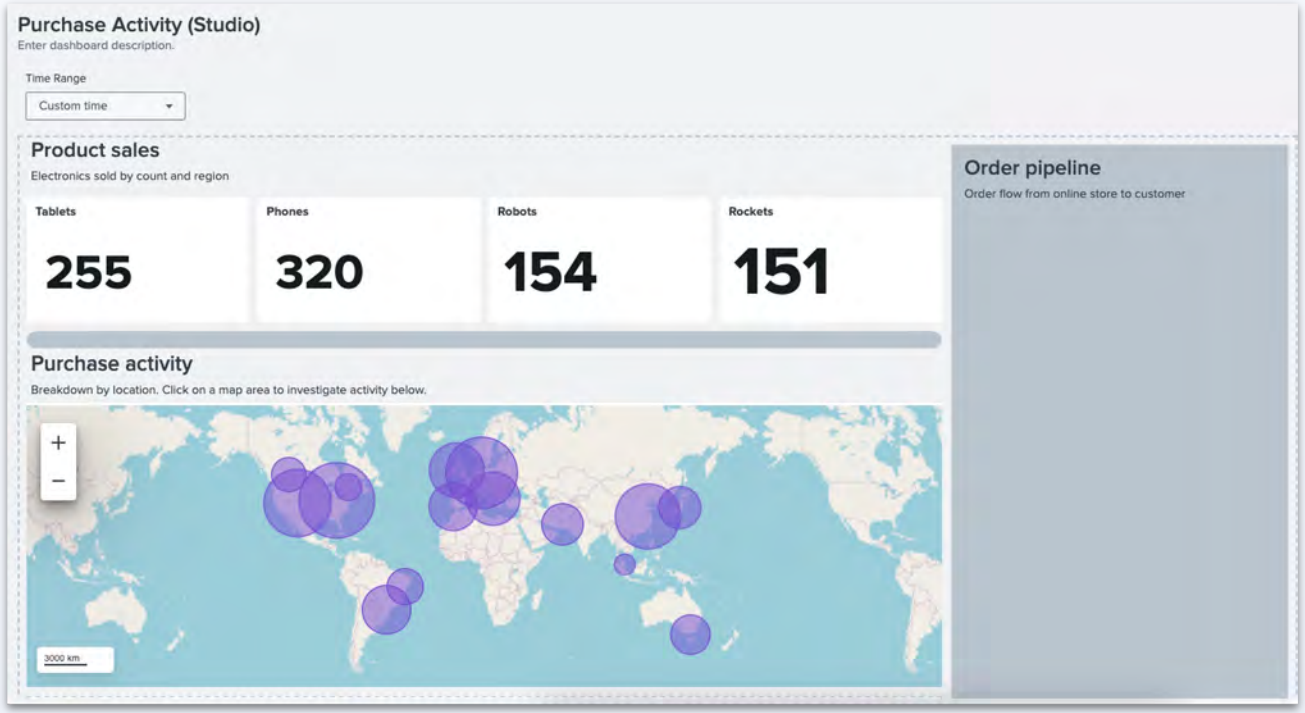
Exercise 4: Set up the conversion funnel



Create a section for the funnel

1. Create the title for the conversion funnel using a Markdown component:
   # Order pipeline
   Order flow from online store to customer
2. Move the Markdown component to the top left of the remaining third of the dashboard.
3. Create a rectangle with a transparent stroke that fills the right side of the dashboard. This will be the background of the funnel.

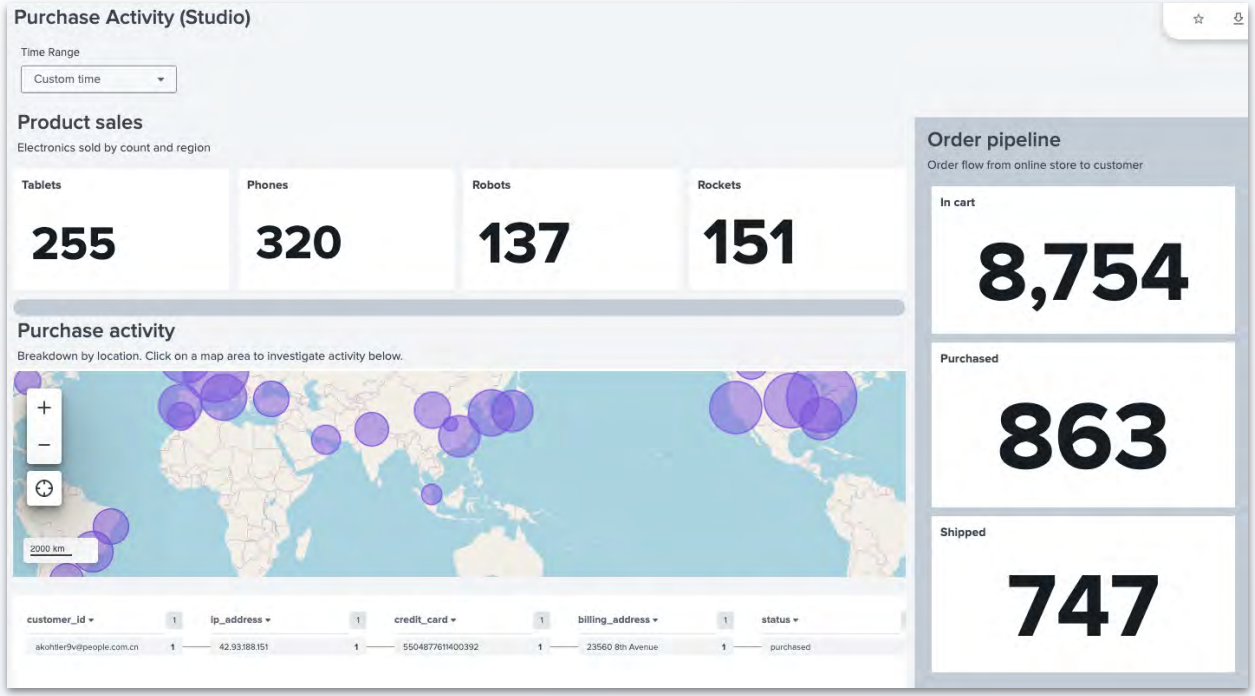




© 2026 Cisco and/or its affiliates. All rights reserved.

Place order status KPIs

4. Assemble the order status KPIs (In cart, Purchased, Shipped) to the conversion funnel.



© 2026 Cisco and/or its affiliates. All rights reserved.

Set up an alternative "details" view

By default, users will see the 3 single value conversion funnel overview. We want to provide users a way to see detailed information about each step.

We will add a button for users to choose between the "Overview" and "Details" views.

We will overlay detailed charts on the conversion funnel that will be hidden when users select "Overview" and visible when users select "Details".

© 2026 Cisco and/or its affiliates. All rights reserved.

Create a button to toggle overview or details

This requires 2 token eval expressions!

Today, you need to configure token eval expressions in the source code. Each token eval expression requires:

- A unique ID, e.g. "expr_1"
- A name for the expression, e.g. "Button label"
- A value (the expression)

While a conditional expression must resolve to a Boolean value (true or false), eval expressions can resolve to strings, numbers, and arrays as well as Boolean values.

© 2026 Cisco and/or its affiliates. All rights reserved.

Create a token eval expression for the button label

We want the button to say "Show details" when showing the overview single value visualizations and say "Show overview" when showing the details table and line chart.

We are going to use a new token, $detailsVisibility$, that when set to true, will show the button label "Show overview".

In the dashboard definition source code, add the expressions stanza with the eval expression:

```
"expressions": {
    "eval": {
      "expr_1": {
        "name": "detailsBtnLabel",
        "value": "$detailsVisibility$ = 'true' ? 'Show overview' : 'Show details'"
      }
    }
  },
```

© 2026 Cisco and/or its affiliates. All rights reserved.

Create a token eval expression for $detailsVisibility$ value

When the user clicks the button, we need to toggle $detailsVisibility$ between true and false.

If $detailsVisibility$ is equal to true, that means we are showing the detailed visualizations (table and line chart) and the button label is "Show overview". Thus, when a user clicks the button, we should set $detailsVisibility$ to false, hide the detailed visualizations, and change the button label to "Show details".

After the first eval expression, add a comma, then add the following expression:

```
"expr_2": {
  "name": "toggleDetails",
  "value": "$detailsVisibility$ = 'true' ? 'false' : 'true'"
}
```

Click **Apply and close**.

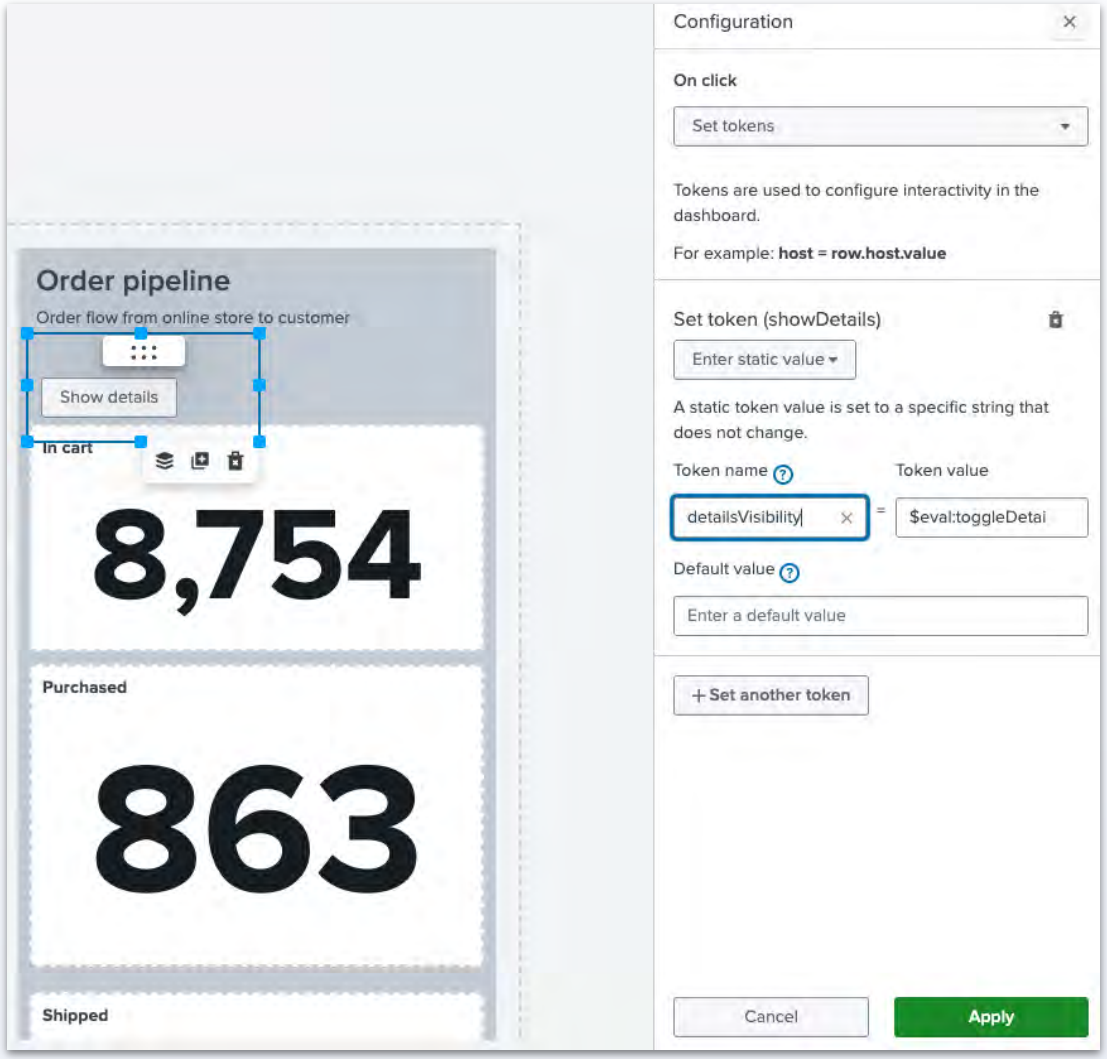
Tip! If your code is underlined with red squiggles and you can't apply your changes, you may have a JSON error. Check that the syntax is all correct – commas, closing brackets, and so on.

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

Set up an alternative "details" view

5. Add a button:
   a. From the inputs dropdown, click **Button**.
   b. Under **Display**, select **In canvas**.
   c. Position above the ‘In cart’ single value visualization.
   d. For **Label**, enter **$eval:detailsBtnLabel$**
   e. Under **Interactions**, click **Add interaction**
   f. From the **On click** dropdown, select **Set tokens**
   g. Configure the following token:
      a. Token name: detailsVisibility
      b. Token value: $eval:toggleDetails$
   h. Click **Apply**, then click the button to test out that its label changes from Show details to Show overview and vice-versa.



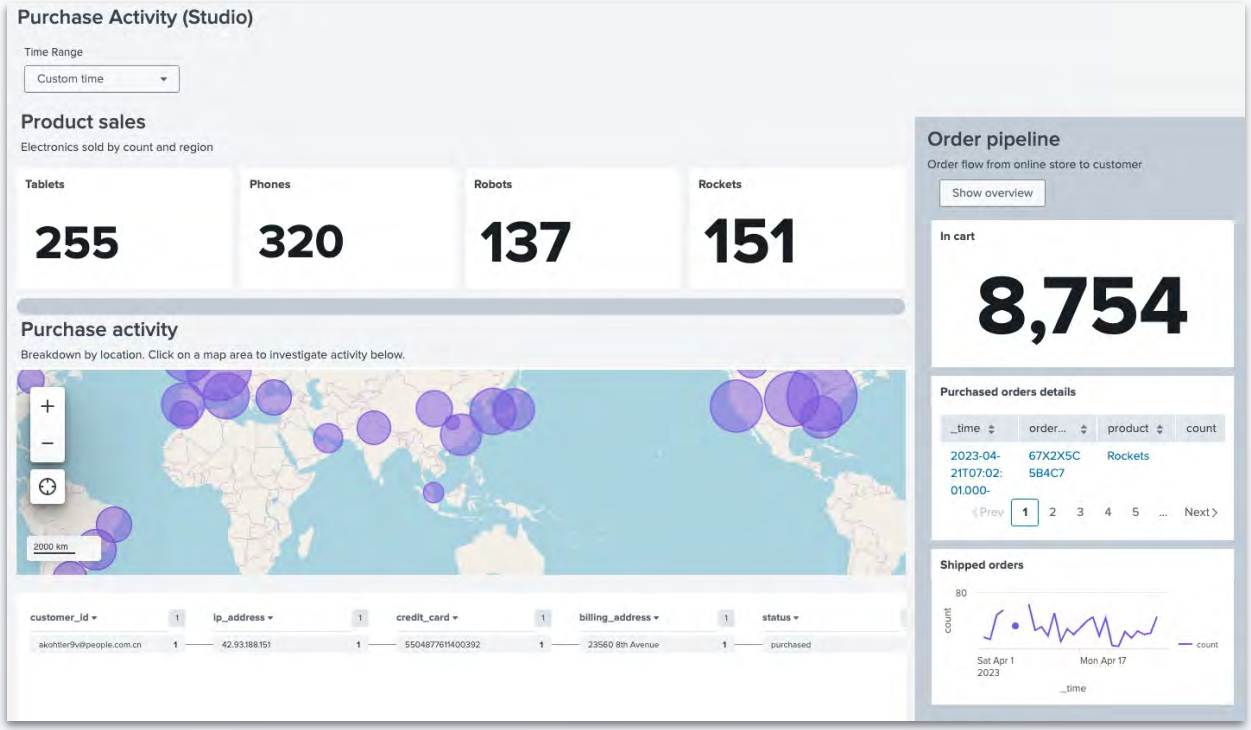
Note: If the button label in the canvas shows **$eval$**, there may be a typo. Ensure that the token name and value are configured to match the expressions you configured in the source code.

© 2026 Cisco and/or its affiliates. All rights reserved.

Set up an alternative "details" view

6. Place the "Purchased orders details" table over the "Purchased" single value.

7. Place "Shipped orders" chart over the "Shipped" single value.



© 2026 Cisco and/or its affiliates. All rights reserved.

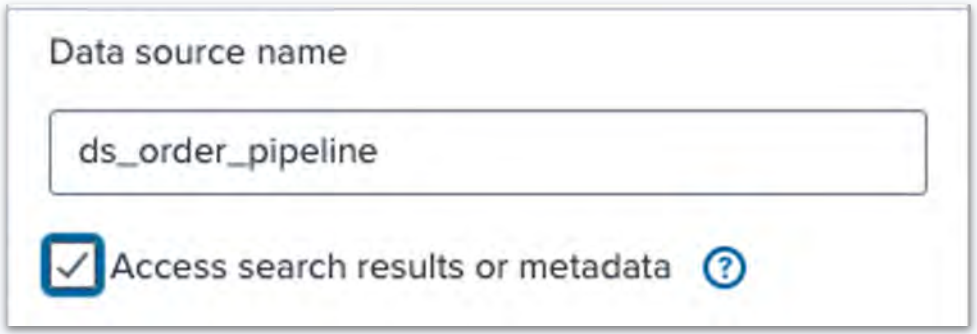
Set up an alternative "details" view

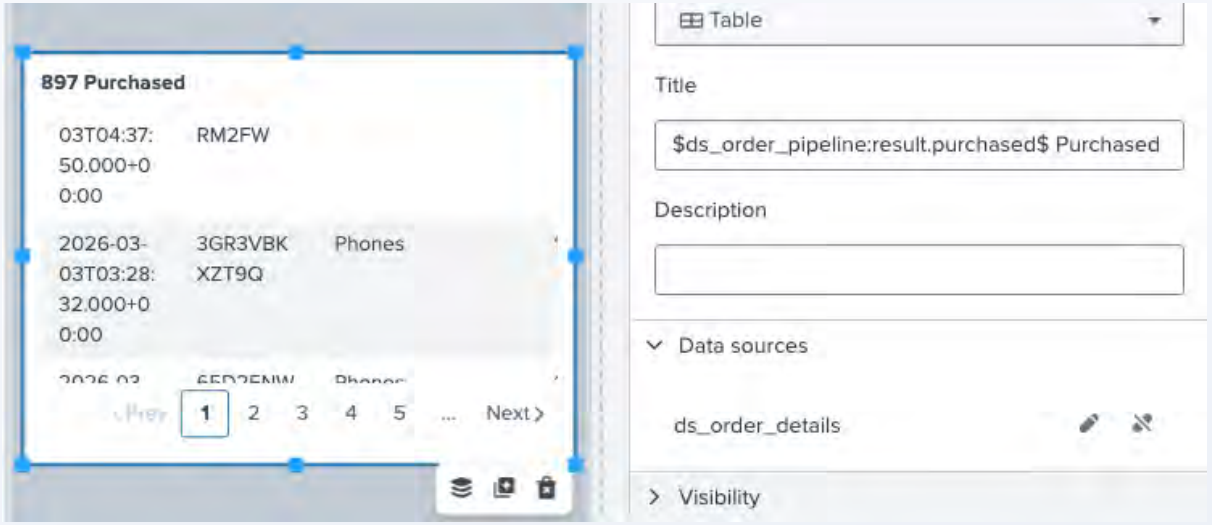
When viewing "details", we can't see the total number of purchased and shipped orders. Let's add them to the viz titles.

10. Navigate to the **ds_order_pipeline** search and check "**Access search results or metadata**". Make sure to select **Apply and close**.

11. Update the Purchased ordered details table's title to: **$ds_order_pipeline:result.purchased$ purchased**

12. Update the Shipped orders line chart's title to: **$ds_order_pipeline:result.shipped$ shipped**





Checkpoint! Save your dashboard periodically so you don't lose your changes.

© 2026 Cisco and/or its affiliates. All rights reserved.

About search-based tokens

Search-based tokens allow you to reference a search's job metadata or the first row of results directly as tokens using the following syntax:

- $Data source name:job.<metadata>$
- $Data source name:result.<fieldname>$

For example, if we had a data source called **Order pipeline**, we could reference:

- Job ID using **$Order pipeline:job.sid$**
- Shipped orders status using **$Order pipeline:result.shipped$**

To learn more, check out the Dashboard Studio docs

© 2026 Cisco and/or its affiliates. All rights reserved.

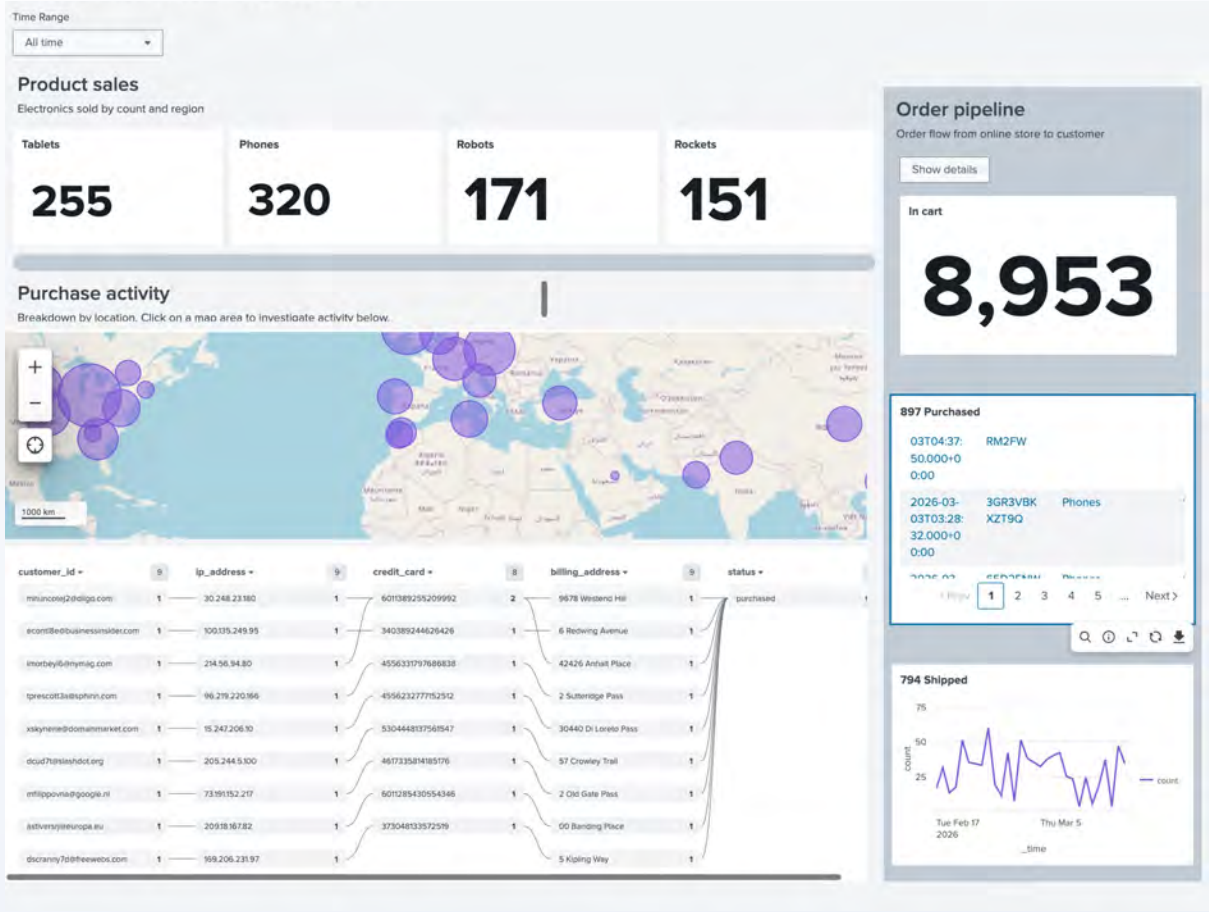
Exercise 5:
Set up advanced conditional visibility

Configure multiple visibility conditions

In Splunk Enterprise 10.2 and Splunk Cloud 10.1.2507, you can configure advanced conditional visibility rules and apply multiple rules per dashboard panel.

We'll add a multiselect that allows the user to filter the visualizations in the **Order pipeline** by order status. We'll configure conditions so that, for example, if the user selects only **In cart** and **Shipped** from the multiselect, the **Purchased** visualizations are hidden. We'll also configure a condition that shows the detail or overview charts based on the button status.

You can configure multiple visibility conditions to replace or enhance the **When data is unavailable, hide element** feature.

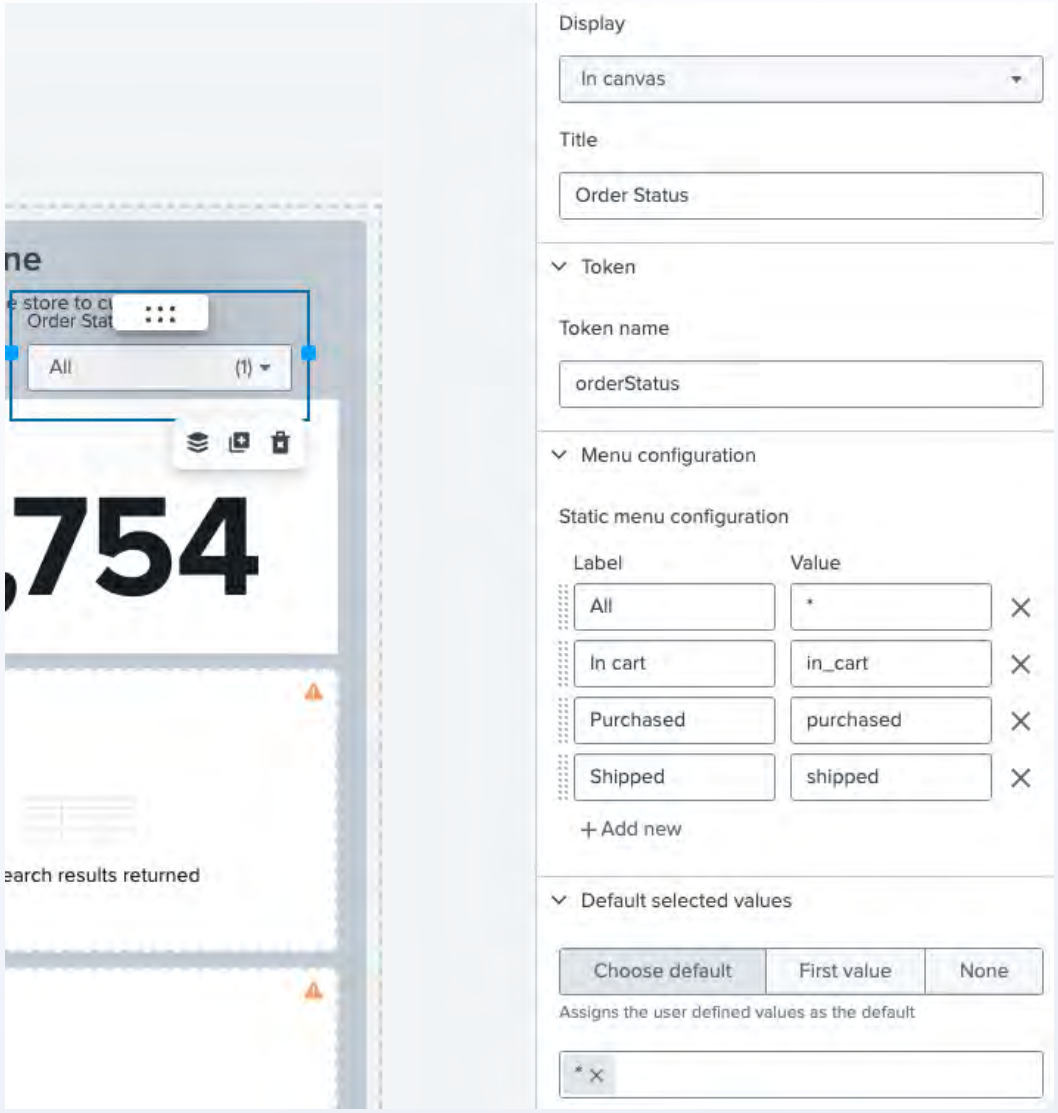


© 2026 Cisco and/or its affiliates. All rights reserved.

Configure a multiselect input

Let's first create the multiselect input to filter visualizations by order status.

1. Add a multiselect input:
   - Display: In canvas. Drag the multiselect beside the Show overview/details button.
   - Title: Order Status
   - Token: orderStatus
   - Menu configuration:
     - Label: All, Value: *
     - Label: In cart, Value: in_cart
     - Label: Purchased, Value: purchased
     - Label: Shipped, Value: shipped
2. Configure * as the only default value.
3. Open the multiselect dropdown and select "Clear all" to remove the item001 and item002 selections



© 2026 Cisco and/or its affiliates. All rights reserved.

Configure multiple visibility conditions

The following slides will provide instructions for configuring the following visibility conditions for the visualizations in the Order pipeline:

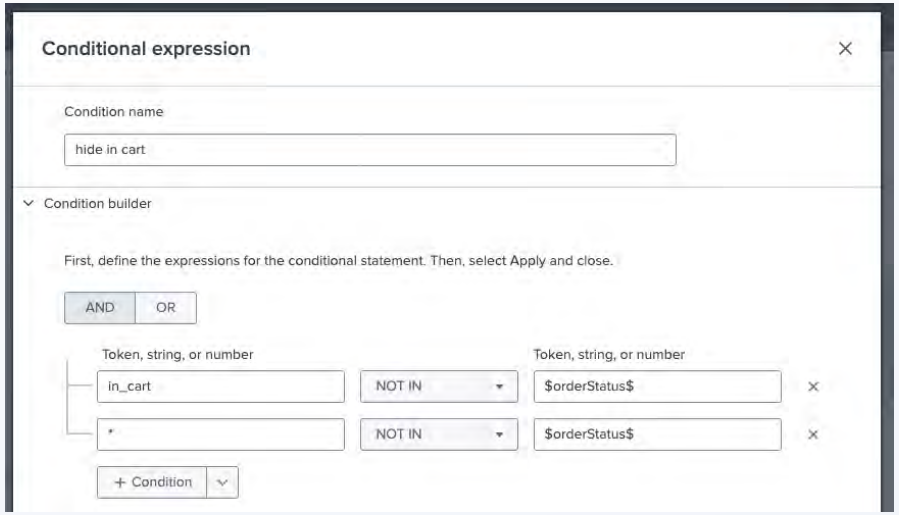
- **In cart**:
  - Hide when neither In cart nor All are selected in the Order Status multiselect
- **Purchased table (863 purchased)**:
  - Hide when neither Purchased nor All are selected in the Order Status multiselect
  - Show when $detailsVisibility$ is true
  - When a dashboard panel has conflicting conditions, it is hidden by default. So, in this case, if both conditions are met, the table is hidden.
- **Purchased single value visualization**:
  - Hide when neither Purchased nor All are selected in the Order Status multiselect
- **Shipped line chart (747 shipped)**:
  - Hide when neither Shipped nor All are selected in the Order Status multiselect
  - Show when $detailsVisibility$ is true
  - When a dashboard panel has conflicting conditions, it is hidden by default. So, in this case, if both conditions are met, the chart is hidden.
- **Shipped single value visualization**:
  - Hide when neither Shipped nor All are selected in the Order Status multiselect

© 2026 Cisco and/or its affiliates. All rights reserved.

Configure the hide in cart condition

Configure a visibility condition for the In cart visualization.

4. For the In cart visualization, under **Visibility > Custom visibility conditions**, click **+ Set up condition**.
   - Click Create condition.
   - Set the **Condition name** as "**hide in cart**".
   - The In cart visualization should be hidden when the user has not selected All or In cart from the multiselect. Add the following conditional statements, and ensure AND is selected so that the condition requires that both statements be fulfilled:
     - in_cart NOT IN $orderStatus$
     - * NOT IN $orderStatus$
   - The second condition is for when the user selects All from the multiselect and $orderStatus$ is equal to *.
   - Click Apply and close.
   - Click the eye icon so that it is crossed out. This means that when the condition is met, the panel will be hidden.





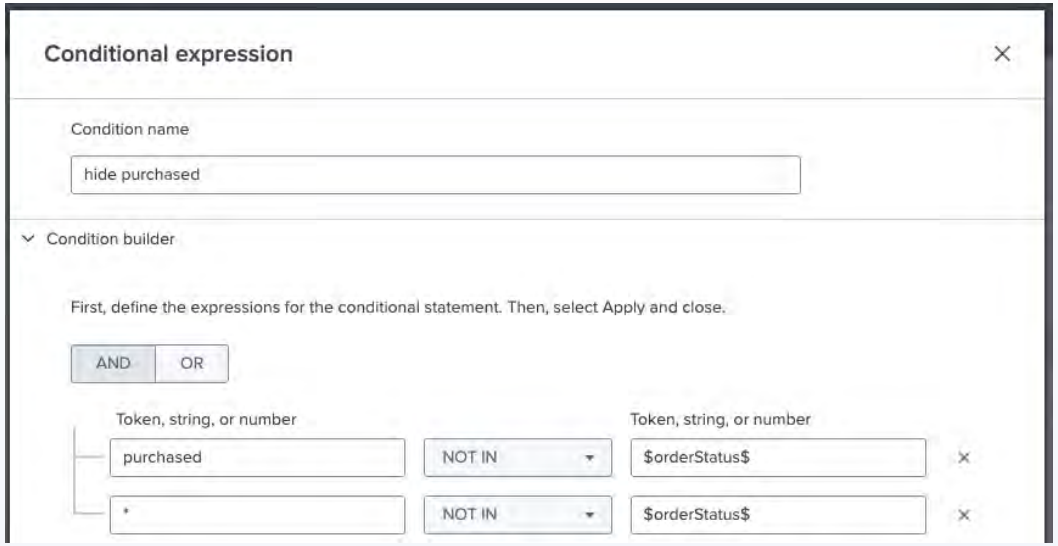
© 2026 Cisco and/or its affiliates. All rights reserved.

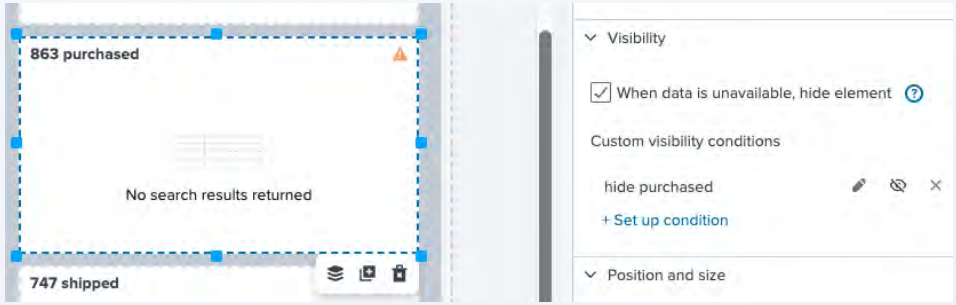
Configure the hide purchased condition

Next, let's configure a visibility condition for the Purchased table (863 purchased).

5. For the Purchased table, create a condition named "hide purchased".

- The Purchased table should be hidden when the user has not selected All or Purchased from the multiselect. Add the following conditional statements, and ensure AND is selected so that the condition requires that both statements are fulfilled:
  - purchased NOT IN $orderStatus$
  - * NOT IN $orderStatus$
- Click the eye icon so that it is crossed out. This means that when the condition is met, the panel will be hidden.





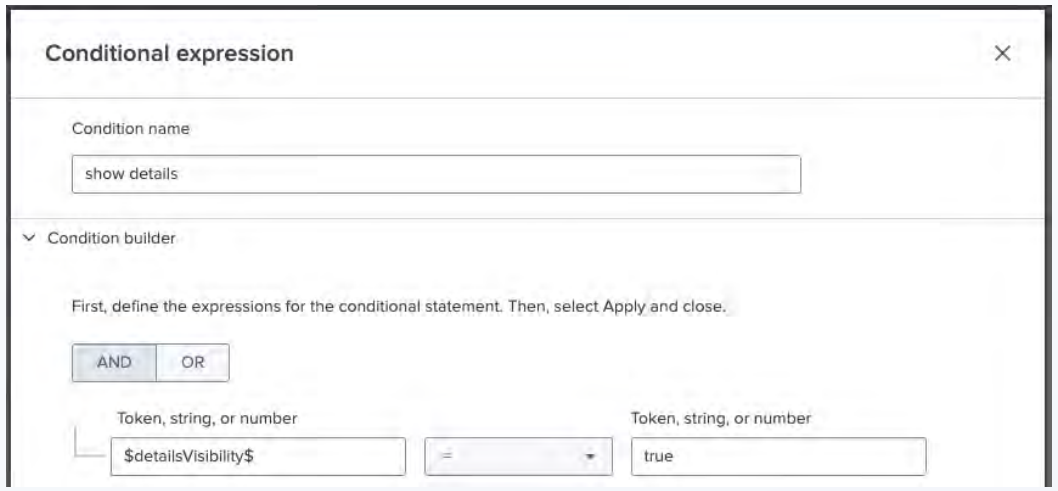
© 2026 Cisco and/or its affiliates. All rights reserved.

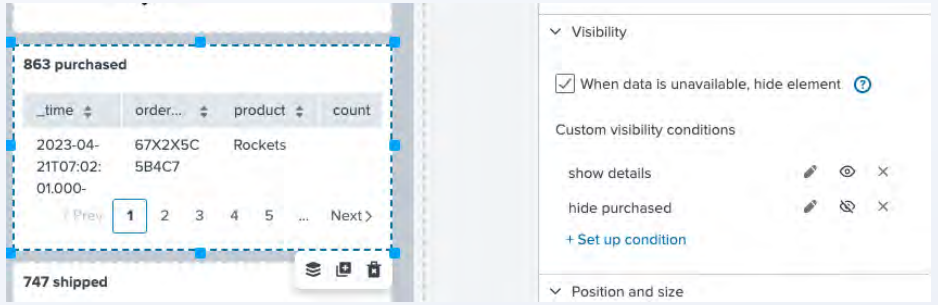
Configure the show details condition

Next, let's configure another visibility condition for the Purchased table (863 purchased).

6. For the Purchased table, create a condition named "show details".

- The Purchased table should display when the user has clicked the button to show details. Add the following conditional statement:
    - $detailsVisibility$ = true
- Leave the eye icon so that it is not crossed out. This means that when the condition is met, the panel will display.





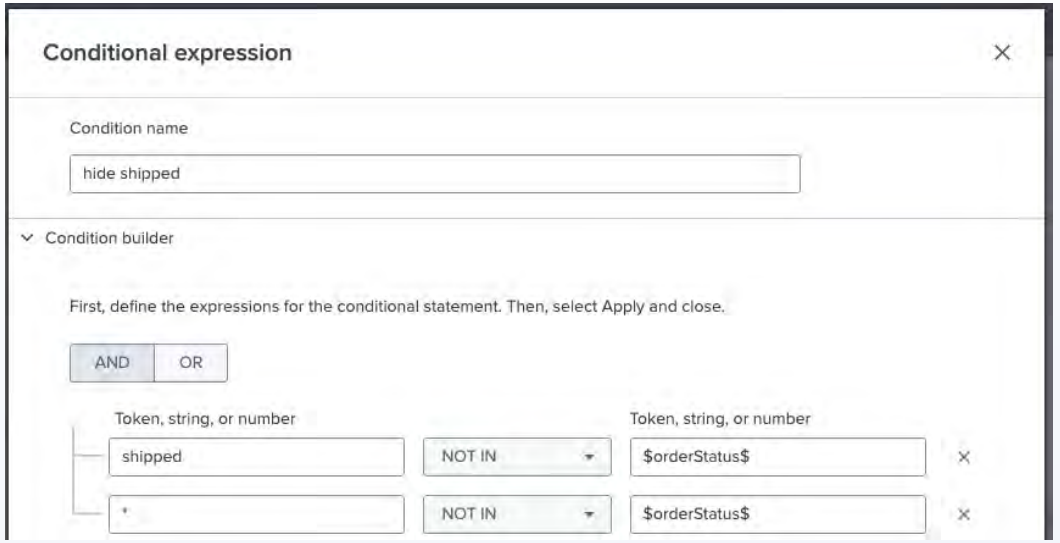
© 2026 Cisco and/or its affiliates. All rights reserved.

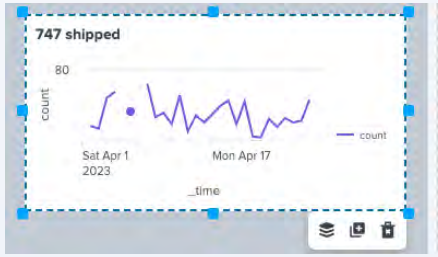
Configure the hide shipped condition

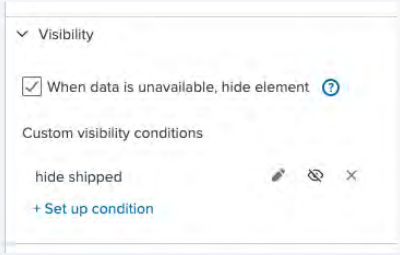
Next, let's configure a visibility condition for the Shipped line chart (747 shipped).

7. For the Shipped line chart, create a condition named "hide shipped".

   - The Shipped line chart should be hidden when the user has not selected All or Shipped from the multiselect. Add the following conditional statements, and ensure AND is selected so that the condition requires that both statements are fulfilled:
     - shipped NOT IN $orderStatus$
     - * NOT IN $order status$
   - Click the eye icon so that it is crossed out. This means that when the condition is met, the panel will be hidden.



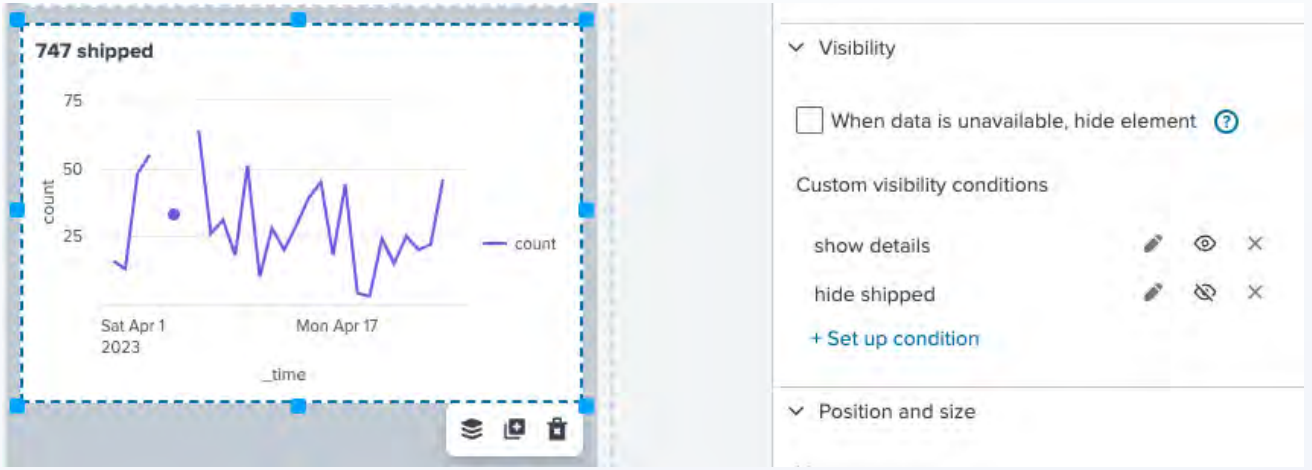




© 2026 Cisco and/or its affiliates. All rights reserved.

Apply visibility conditions

Now we'll apply the show details conditions that we already created to the Shipped line chart (747 shipped). This ensures that the Shipped line chart displays when the user clicks the button to show details.



8. Select the Shipped line chart, then under **Visibility**, select **+ Set up condition**, then the **show details** condition that we created earlier. Leave the eye icon so that it is not crossed out.

© 2026 Cisco and/or its affiliates. All rights reserved.

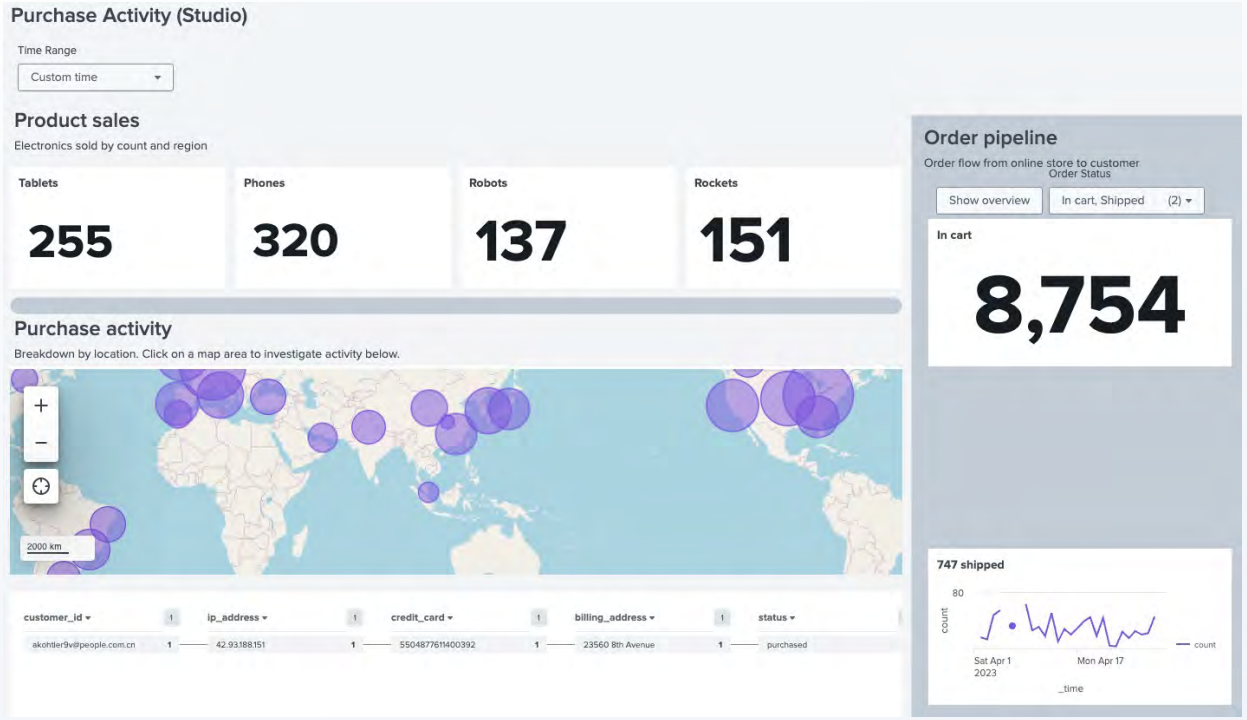
Apply visibility conditions

For the single values, we'll configure conditions so that they're hidden when the user hasn't selected to see **All** or the relevant status (**Purchased** or **Shipped**) from the multiselect.

To reach the single value visualizations, you'll have to move the Purchased table and Shipped line chart out of the way.

9. For the Purchased single value visualization, under **Visibility**, configure the **hide purchased** (with the eye crossed out).
10. For the Shipped single value visualization, under **Visibility**, configure the **hide shipped** (with the eye crossed out).

Go to View mode and switch between the overview and details views and different selections in the Order Status multiselect to test visibility.



© 2026 Cisco and/or its affiliates. All rights reserved.

About Dynamic Options Syntax

What is Dynamic Options Syntax (DOS)?

Also known as Splunk visualizations' domain-specific language (DSL)

DOS enables precise customization during configuration of components in Studio.

```
"someOption": "> someDataSource | someSelectors(...) | someFormatter(...)"
```

Always starts with ">"

Sections separated using "|"

For example, instead of using a static hex code such as "#FFFFFF" to color the major value in a single value component, you can use dynamically-powered range value thresholds: 0-40=red, 40-70=yellow, 70-100=green that depend on a specified field hidden from view.

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

What is Dynamic Options Syntax (DOS)?

Also known as Splunk visualizations' domain-specific language (DSL)

DOS enables precise customization during configuration of components in Studio.

```
"someOption": "> someDataSource | someSelectors(...) | someFormatter(...)"
```

majorColor

The configuration option is the aspect of the visualization component you'd like to change, for example:

- The major value's color in a single value visualization

– The y-axis' field selections in a line chart

© 2026 Cisco and/or its affiliates. All rights reserved.

What is Dynamic Options Syntax (DOS)?

Also known as Splunk visualizations' domain-specific language (DSL)

DOS enables precise customization during configuration of components in Studio.

```
"someOption": "> someDataSource | someSelectors(...) | someFormatter(...)"
```

primary

The first part of the DOS string is the source of data or values you'd like to to dynamically power the configuration, for example:

– The primary, or annotation data source

– Another configuration option to use the exact same value

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

What is Dynamic Options Syntax (DOS)?

Also known as Splunk visualizations' domain-specific language (DSL)

DOS enables precise customization during configuration of components in Studio

```
"someOption": "> someDataSource | someSelectors(...) | someFormatter(...)"
```



The second part of the DOS string is often a filter for the data source (and is sometimes optional), for example:

– Selecting the field named "count", and then filtering further to select the "count" column's last point

– Selecting only the numeric columns

Note: these require single quotation marks `'` or escaped double quotation marks `\"` around the selection parameters

© 2026 Cisco and/or its affiliates. All rights reserved.

What is Dynamic Options Syntax (DOS)?

Also known as Splunk visualizations' domain-specific language (DSL)

DOS enables precise customization during configuration of components in Studio.

```
"someOption": "> someDataSource | someSelectors(...) | someFormatter(...)"
```

```
rangeValue(someColorContext
)
```

The third part of the DOS string is often a formatter for the data source (and is sometimes optional), for example:

- Using range value thresholds to color, which are referenced using a variable named defined in the context section of the stanza
- Adding a unit prefix

© 2026 Cisco and/or its affiliates. All rights reserved.

Dynamic Options Syntax (DOS) Example

Here's what the example we just walked through would look like in source:

```
"viz_fWZQq3mP": {
    "type": "splunk.singlevalue",
    "options": {
        "majorColor": "> primary | seriesByName('count') | lastPoint() | rangeValue(majorColorEditorConfig)"
    },
    "dataSources": {
        "primary": "ds_AZuF9nvk"
    },
    "context": {
        "majorColorEditorConfig": [
            {
                "to": 0,
                "value": "#D41F1F"
            },
            {
                "from": 0,
                "value": "#118832"
            }
        ]
```

© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 6:
Color table columns by hidden values with DOS

Color table columns by hidden values

We want to color the **count** column red or green depending on whether there are more of that product in stock, which is driven by the field **_has_stock.**



_has_stock is hidden by default because the underscore (_) indicates that it should be treated as an internal field, and under **Data display**, **Internal fields** is unchecked by default.

You can check the box to see **_has_stock** in the table.

Note: You may need to adjust the table columns or scroll horizontally to view all columns. Ensure "Details" is selected in the dropdown

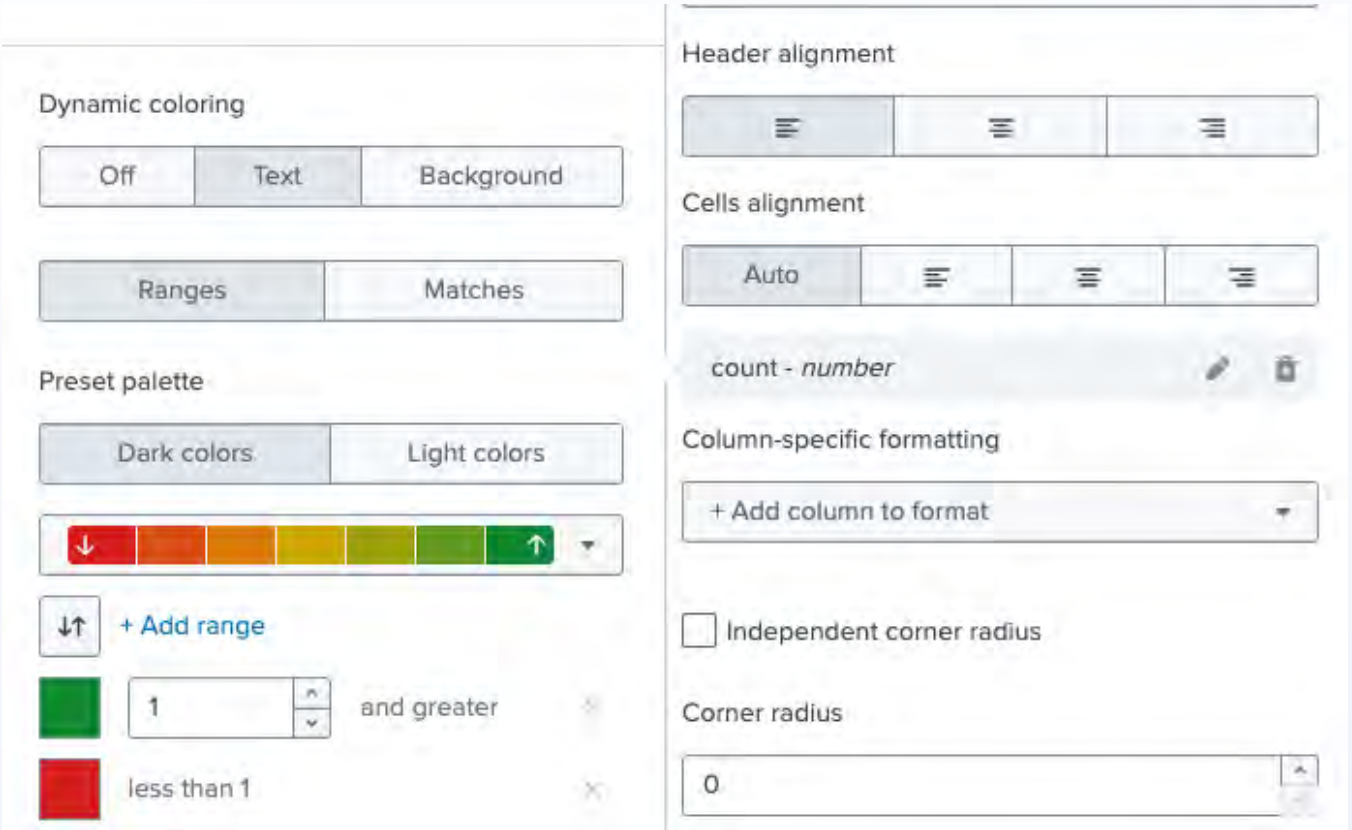
the values in the **count** column

© 2026 Cisco and/or its affiliates. All rights reserved.

Applying color to "count"

As a best practice, configure as much as possible in the UI **first** before modifying the source code to minimize the chances of syntax errors.

1. Under **Column-specific formatting**, add the **count** column to format.
2. For the **count** column, click the pencil icon.
3. Under **Dynamic coloring,** select **Text.**
4. Apply ranges so that **1 and greater** is **green**, and **less than 1** is **red.**



© 2026 Cisco and/or its affiliates. All rights reserved.

Update the DOS to use _has_stock

So far we've configured the **count** column to be green when **count** is greater than 0 and red when 0.

What we want is to configure **count** to be green when **_has_stock** is greater than 0 and red when 0.

5. In source, under **"options"** > **"columnFormat"** > **"count"** > **"rowColors",** change **count** to **_has_stock**

Your source code should look like this:

```
2     "type": "splunk.table",
3     "options": {
4         "columnFormat": {
5             "count": {
6                 "data": "> table | seriesByName(\"count\") | formatByType(countColumnFormatEditorConfig)",
7                 "rowColors": "> table | seriesByName(\"_has_stock\") | rangeValue(countRowColorsEditorConfig)"
8             }
```

count powers the data configuration option

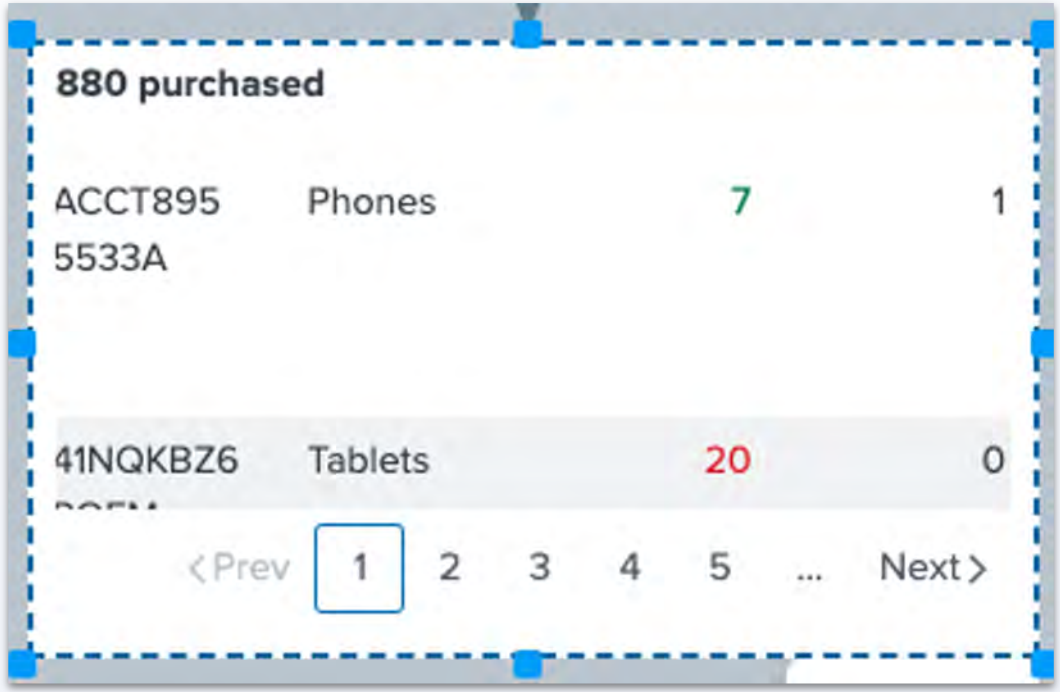
_has_stock powers the rowColors configuration option

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

Update the DOS to use _has_stock

Your table should now have values in **count** that are red or green, depending on the value of the **_has_stock** field

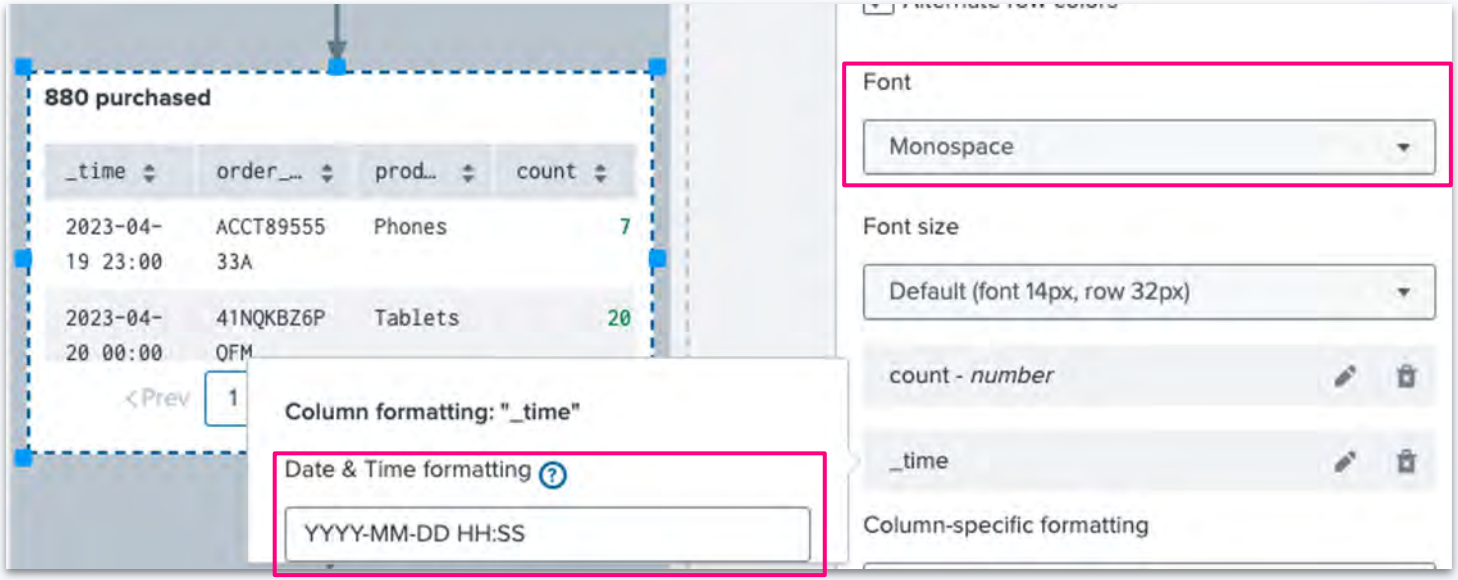


© 2026 Cisco and/or its affiliates. All rights reserved.

Finish formatting the table

6. Under **Color and style,** change **Font** to **Monospace**.
7. Under **Column-specific formatting**, select **_time** and apply the format:

YYYY-MM-DD HH:SS

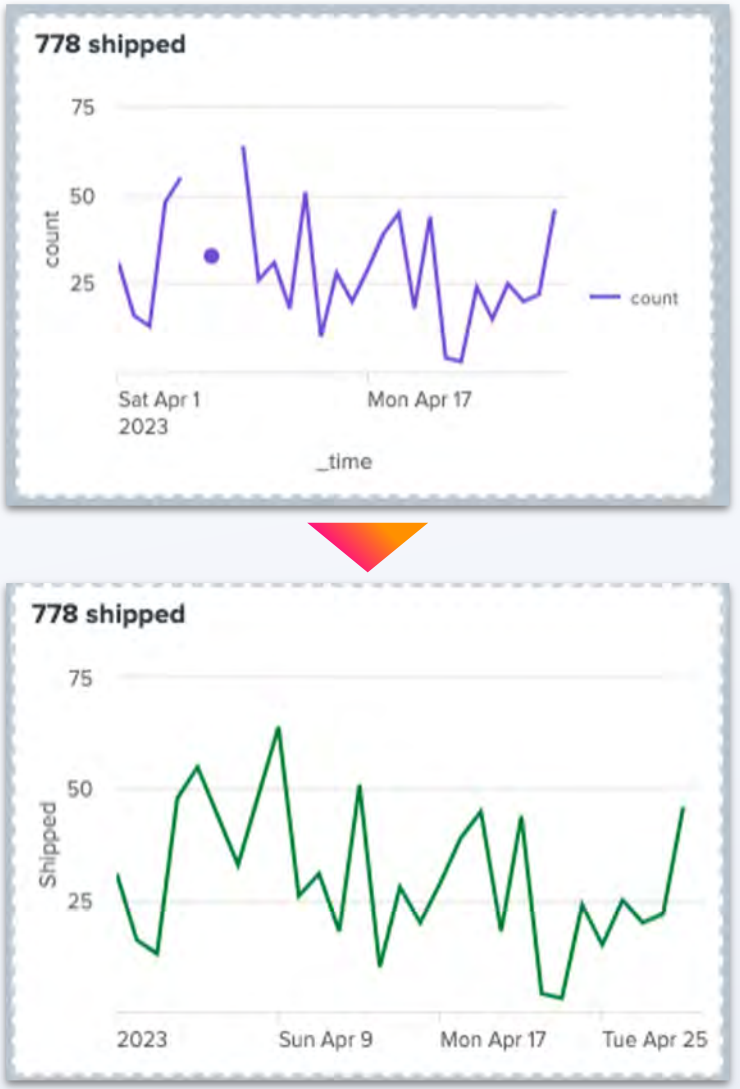


© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 7:
Add secondary data sources to line charts with DOS

Beautify the "Shipped orders" line chart

1. Under **Data display**, change **Null value display** to "**Connect**"
2. Under **Color and style**, add a series to **Series colors by field name**. Select the field name **count** and apply the color **#118832**
3. Under **Legend** change **Legend display** to "**Off**"
4. Under **X-axis grid and labels**, deselect **Axis title**, and change **Number of time label parts** to 1
5. Under **Y-axis grid and labels,** specify **Axis title text** as "**Shipped**".



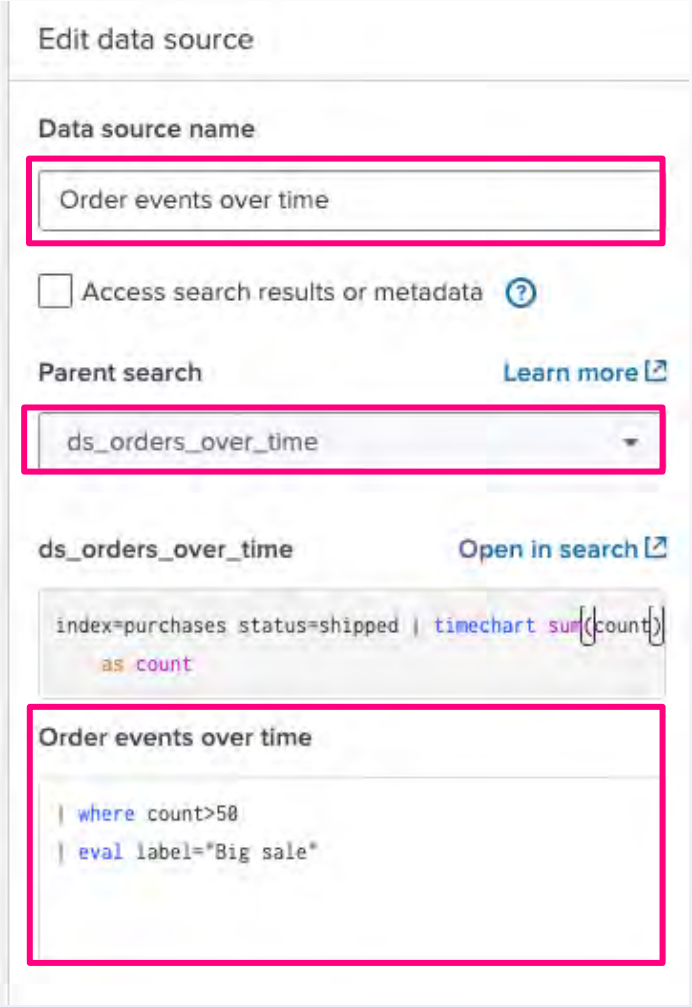
© 2026 Cisco and/or its affiliates. All rights reserved.

Add an annotation data source

Use annotations to add context to your charts. Ensure your annotation data source returns fields that corresponds to the chart's x-axis (usually _time), the label you want to display, and, optionally, a color field.

6. Select the line chart. Under **Data sources**, click on
   + Set up annotation data source
7. Create a new chain search and click on **Apply & close**
   a. Name: Order events over time
   b. Parent search: ds_orders_over_time
   c. Chain SPL:

```
| where count>50
| eval label="Big sale"
```



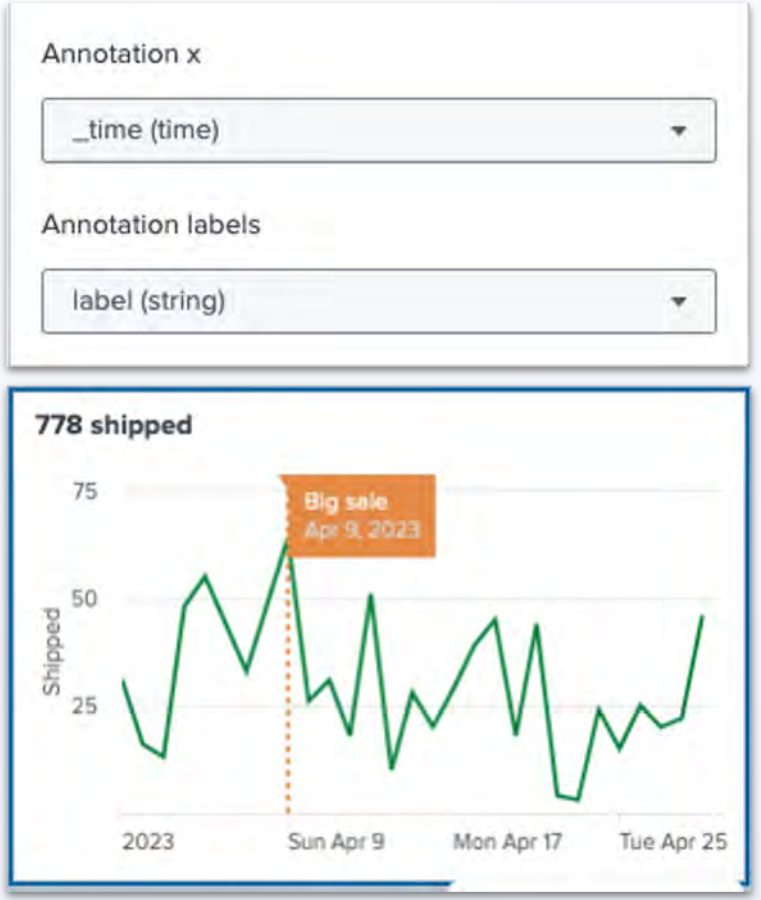
© 2026 Cisco and/or its affiliates. All rights reserved.

Configure the chart annotation

Under **Data configurations:**

8. Select **Annotation x** to be "**_time (time)**"
9. Select **Annotation labels** to be "**label (string)**"

Now in **View** mode, you can hover on the annotation to see the label

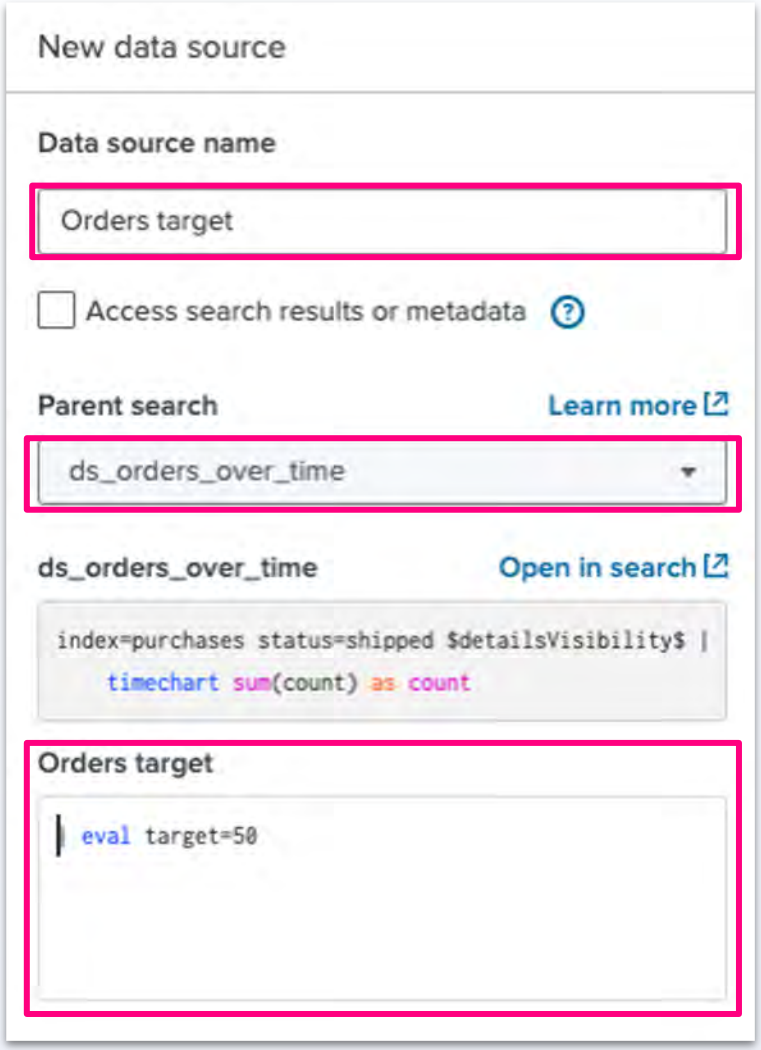


© 2026 Cisco and/or its affiliates. All rights reserved.

Create a search for "target" orders

We want to see whether our shipped orders are meeting our targets, so we will overlay a horizontal line on the line chart as a secondary data source.

10. Navigate to the Data source overview
11. Create a new chain search:
    a. Name: Orders target
    b. Parent search: ds_orders_over_time
    c. Chain SPL: `| eval target=50`
12. Open the code editor and copy the data source ID
13. Click on **Apply and close**.



© 2026 Cisco and/or its affiliates. All rights reserved.

Use DOS to add target data source

Let's add the target as a secondary data source and display it using the y2 axis.

14. Select the line chart and open the code editor
15. Under **"dataSources"** add the ID you just copied from the last step:

```
"secondary": "ds_<your ID>"
```

14. Under **"options"** add:

```
"y2": "> secondary | frameBySeriesNames('target')"
```




```
14        "xAxisTitleVisibility": "hide",
15        "xAxisMaxLabelParts": 1,
16        "yAxisTitleText": "Shipped",
17        "annotationX": "> annotation | seriesByName('_time')",
18        "annotationLabel": "> annotation | seriesByName('label')",
19        "y2": "> secondary | frameBySeriesNames('target')"
20    },
21    "dataSources": {
22        "primary": "ds_orders_over_time",
23        "annotation": "ds_OPHAbljY",
24        "secondary": "ds_TEn8e0aK"
25    },
```

Checkpoint! Save your dashboard periodically so you don't lose your changes.

© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 8:
Color map bubbles with DOS

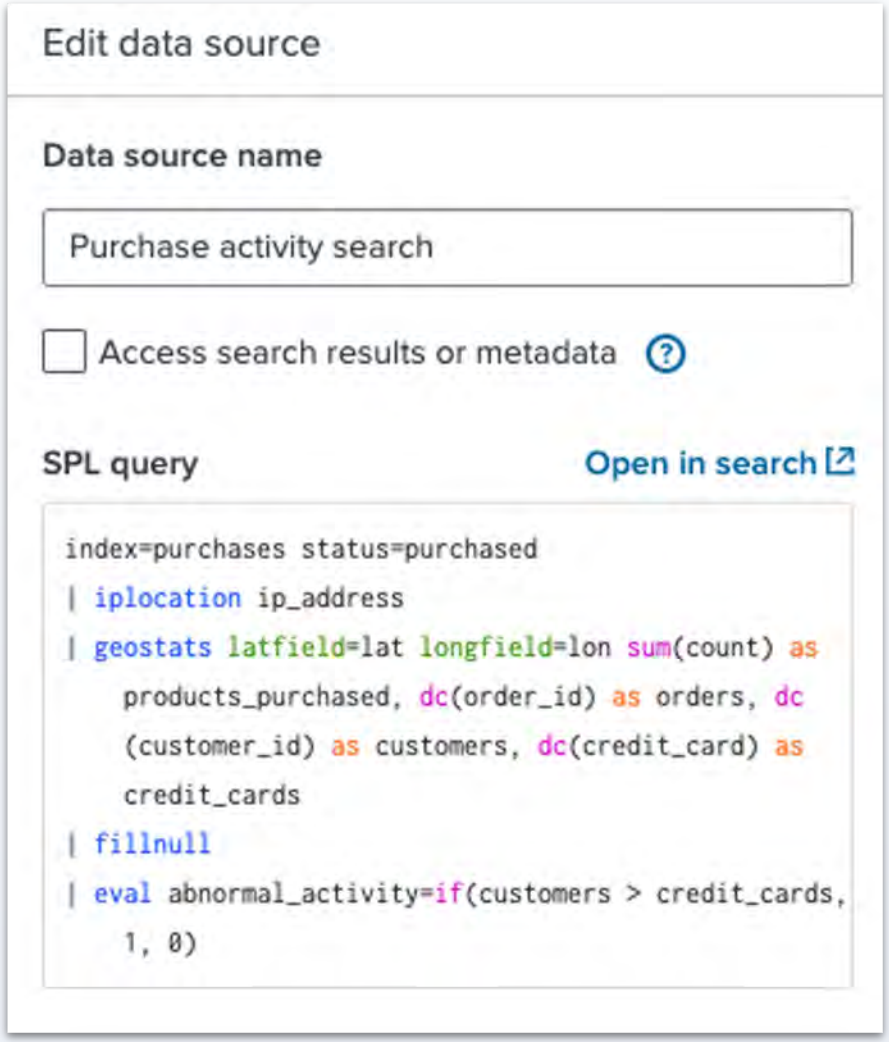
Search for abnormal activity

We want to color our map bubbles green for normal credit card activity, and red for abnormal credit card activity (more customers than credit cards in use)

1. Update the map data source to create a new field to designate abnormal activity by appending:

```
| fillnull
| eval abnormal_activity=if(customers > credit_cards, 1, 0)
```

2. Click **Apply and close**.



© 2026 Cisco and/or its affiliates. All rights reserved.

Define coloring rules in source

3. Under **"context"** create a section called **thresholdConfig:**

```
"context": {
   "thresholdConfig": [
       {
           "to": 1,
           "value": "green"
       },
       {
           "from": 1,
           "value": "red"
       }
   ]
}
```

© 2026 Cisco and/or its affiliates. All rights reserved.

Apply colors using DOS

4. Under **"options"** under **"layers",** inside the curly braces, add the following:

```
"dataColors": "> primary | seriesByName('abnormal_activity') | rangeValue(thresholdConfig)",
```

Option name to color map bubbles

Reference the field that will determine the color

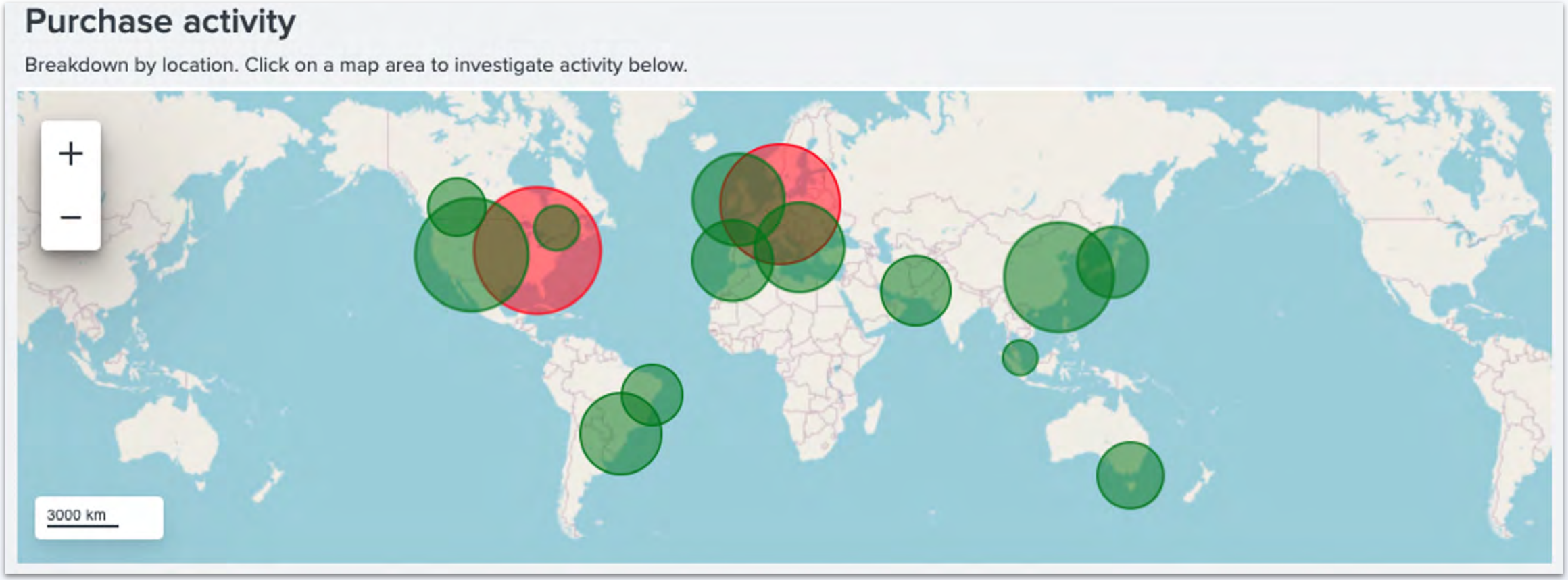
Format using rangeValue, as defined in thresholdConfig

```
],
"layers": [
    {
        "dataColors":">primary | seriesByName('abnormal_activity') | rangeValue(thresholdConfig)",
        "additionalTooltipFields": [
            "credit_cards",
            "customers"
        ],
```

© 2026 Cisco and/or its affiliates. All rights reserved.

cisco

Your map should now be colored like this



© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 9:
Add custom SVGs

What is an SVG?

More documentation about SVGs can be found here

- Scalable Vector Graphics (SVG) is an XML-based markup language for describing two-dimensional based vector graphics
- SVGs can be used for dynamic visualizations in Dashboard Studio, for example:
  - Dynamically coloring a floor plan based on occupancy
  - Interactions like hovering over an SVG to display more information
  - If you can dream it, you can probably build it
- There are many online resources and tools to help you create your own SVGs, which you can then add to Dashboard Studio

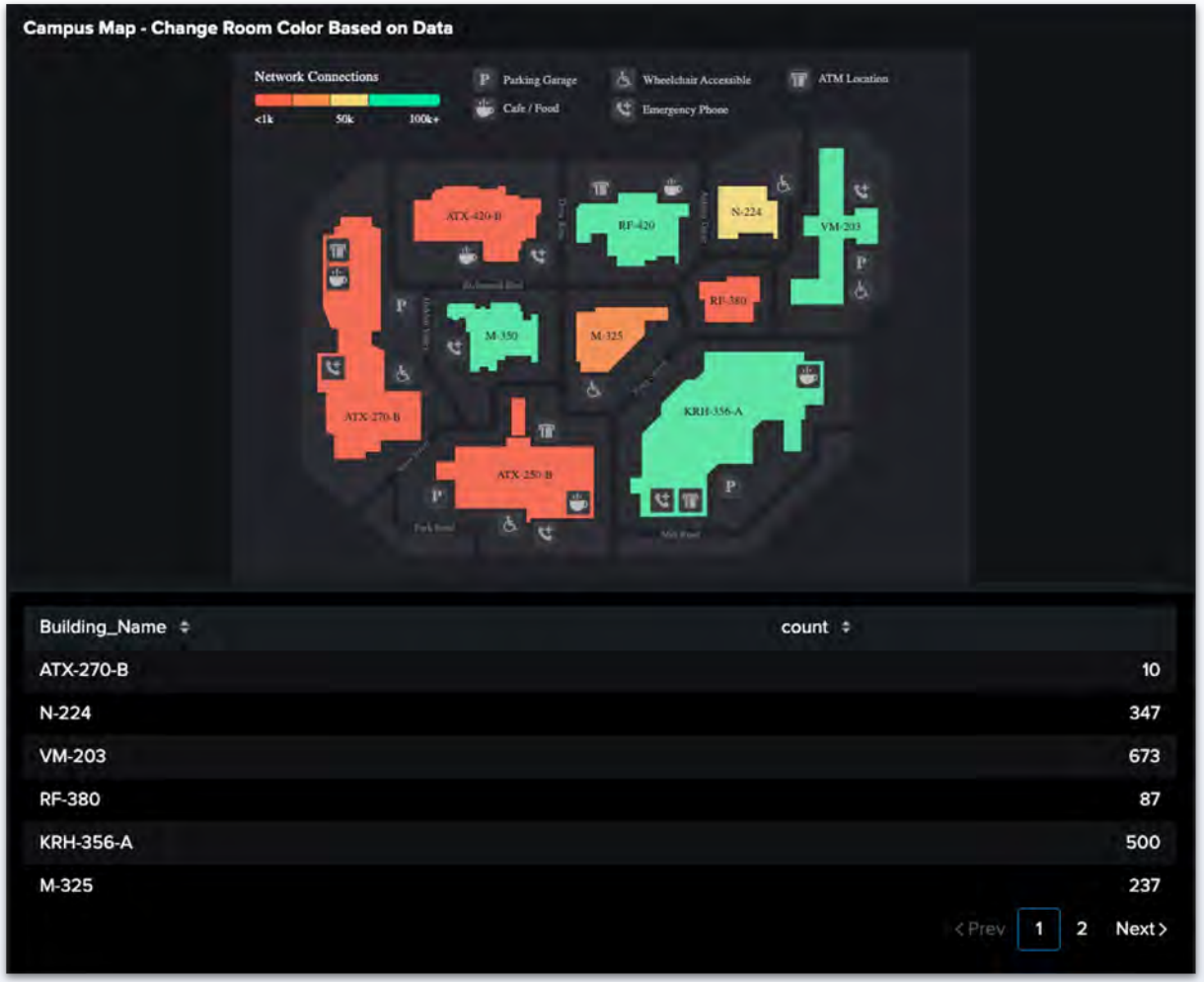
© 2026 Cisco and/or its affiliates. All rights reserved.

SVGs in the Examples Hub

You can find multiple SVG examples in the Examples Hub, such as this campus map.

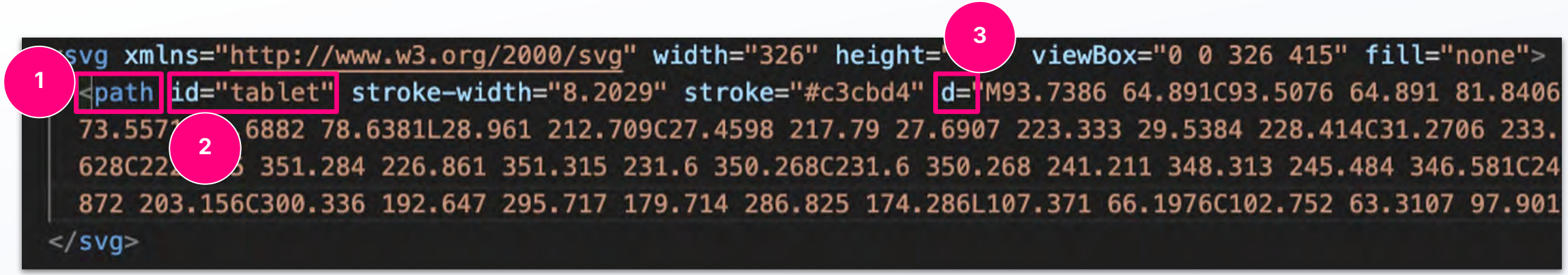
The campus map is an SVG, where each building can be dynamically colored based on search results.

In this example, the buildings are colored based on how many active connections there are per building.



© 2026 Cisco and/or its affiliates. All rights reserved.

Anatomy of an SVG



1. The **`<path>`** element is where we will define the shape (see docs for more details)
2. The **`id`** attribute assigns a unique name to an element, which should map to a search result field name to be able to color dynamically (see previous slide example)
3. The **`d`** attribute is what actually defines the path to be drawn

© 2026 Cisco and/or its affiliates. All rights reserved.

Options for adding SVGs to dashboards

Upload your SVG	Reference via URL	Embed inline SVG
Best when: • Your search will return fields that map to the SVG path ids • You don't need to move your dashboard to another environment	**Best when:** • You want to reuse an SVG in different environments • For Splunk Cloud 9.2.2403 or Splunk Enterprise 9.3 or higher, you can upload to your app's static folder	**Best when:** • You want to reuse an SVG in different environments • You want to embed token values in your SVG definition
Gotchas to look out for: • Uploaded SVGs are placed in the kv-store, so will not transfer if you move your dashboard	**Gotchas to look out for:** • For custom SVGs before Splunk Cloud 9.2.2403 or Splunk Enterprise 9.3, you'll need a webserver to host the SVG	**Gotchas to look out for:** • Most difficult set up • Difficult to read in the source editor

© 2026 Cisco and/or its affiliates. All rights reserved.

Using inline SVGs

- To embed an in-line SVG, you will put the entire SVG definition into the **"svg"** option of your visualization
- These SVGs must be **escaped**, meaning they cannot have new lines characters or double quotes.
  - You can use find and replace in a text editor to replace **\n** with **nothing** and replace **"** with **\"**
- Note that if you have an escaped SVG definition, it will technically not be valid per validation rules, but it will work in Dashboard Studio
- In the next exercise, we will add custom, escaped SVGs to our dashboard

© 2026 Cisco and/or its affiliates. All rights reserved.

Add inline SVGs to your dashboard

We are going to add product SVGs that will dynamically fill with a purple color as more product is sold.



© 2026 Cisco and/or its affiliates. All rights reserved.

Create the inventory data source

1. Create a new search that returns what % of product is left in inventory:
   a. Name: **Inventory totals**
   b. SPL:

```
| makeresults count=1
| eval inventory_tablets=400
| eval inventory_phones=400
| eval inventory_robots=250
| eval inventory_rockets=250
| eval current_tablets=
  (inventory_tablets-$ds_purchases_by_product:result.Tablets$)/inventory_tablets*100
| eval current_phones=(inventory_phones-
$ds_purchases_by_product:result.Phones$)/inventory_phones*100 | eval
current_robots=(inventory_robots-$ds_purchases_by_product:result.Robots$)/inventory_robots*100
| eval current_rockets=
  (inventory_rockets-$ds_purchases_by_product:result.Rockets$)/inventory_rockets*100
| table _time current_tablets current_phones current_robots current_rockets
```

   c. Enable **Access search results or metadata**.

Click **Apply and close.**

© 2026 Cisco and/or its affiliates. All rights reserved.

Create the inventory data source

You may have noticed that the search that we just created uses tokens from the search **ds_purchases_by_product** (example: $ds_purchases_by_product:result.Tablets$)

To use those search results, we need to enable **Access search results or metadata** on **ds_purchases_by_product.**

2. Edit the **ds_purchases_by_product** data source and select **Access search results or metadata.**
3. Click **Apply and close**.

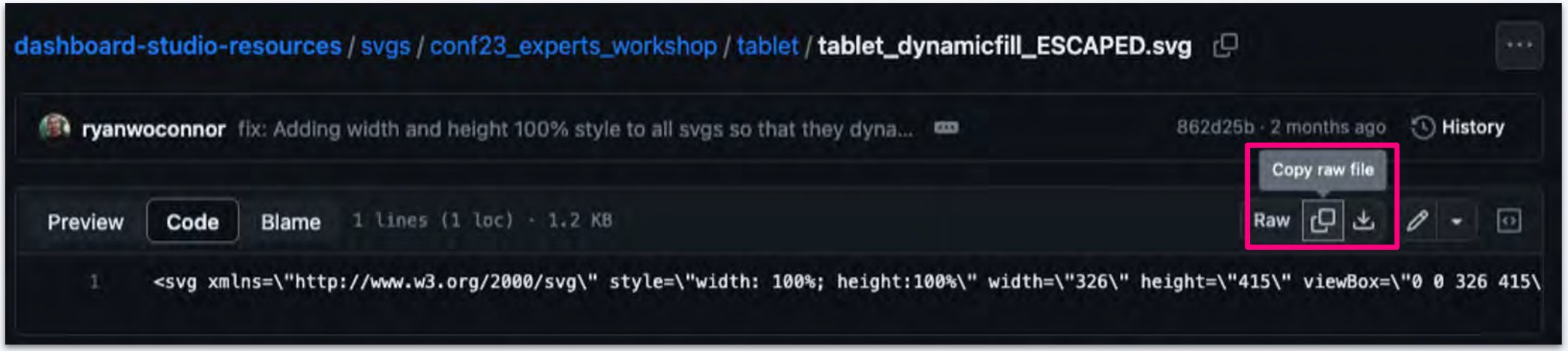
Note: You may need to save and refresh the dashboard to see the SVGs fill.

© 2026 Cisco and/or its affiliates. All rights reserved.

Copy the SVG definition

You will find the definitions for each of the SVGs in our public GitHub repo.
For the sake of today's exercise, here are the links for Tablet, Phone, Robot, and Rocket.

4. Copy the raw SVG code for the Tablet SVG using the **Copy raw file** button:



© 2026 Cisco and/or its affiliates. All rights reserved.

Add an SVG to your dashboard

5. Add a Choropleth SVG and place it next to a single value KPI.
6. When prompted to select a data source, select **Cancel.**
7. Open the Code editor and under **"options"**, add:

   `"svg": "<copied file>"`

8. Repeat these steps for the other products: **Phone**, **Robot,** and **Rocket.**



Checkpoint! Save your dashboard periodically so you don't lose your changes.

© 2026 Cisco and/or its affiliates. All rights reserved.

Let's breakdown the dynamic fill

```
<svg xmlns="http://www.w3.org/2000/svg" width="326" height="415" viewBox="0 0 326 415" fill="none">
  <defs>
    <linearGradient x1="0" x2="0" y1="1" y2="0" id="fill_tablet" fill="#FFF">
      <stop offset="0%" stop-opacity="1" stop-color="#7B56DB"/>
      <stop offset="$Inventory totals:result.current_tablets$%" stop-opacity="1" stop-color="#7B56DB"/>
      <stop offset="$Inventory totals:result.current_tablets$%" stop-opacity="1" stop-color="#FFF"/>
      <stop offset="100%" stop-opacity="1" stop-color="#FFF"/>
    </linearGradient>
  </defs>
  <path id="tablet" stroke-width="8. fill="url(#fill_tablet)" stroke="#c3cbd4" d="M93.7386 64.891C93.5076 64.891 81.8406 67.9618 80.6954 68.
  2763C76.7167 69.6896 73.3049 73.5571 71.6882 78.6381L28.961 212.709C27.4598 217.79 27.6907 223.333 29.5384 228.414C31.2706 233.496 34.6195 237.
  884 39.0077 240.54L218.462 348.628C222.735 351.284 226.861 351.315 231.6 350.268C231.6 350.268 241.211 348.313 245.484 346.581C249.295 344.849
  252.528 341.269 254.145 336.188L296.872 203.156C300.336 192.647 295.717 179.714 286.825 174.286L107.371 66.1976C102.752 63.3107 97.9018 63.932
  93.7386 64.891Z"/>
</svg>
```

1
2

1. Under <defs>, we're defining a HYPERLINK "https://developer.mozilla.org/en-US/docs/Web/SVG/Element/linearGradient"t with the id="fill_tablet", which is referenced later in <path

2. This is where we are defining the linear gradient that will fill the SVG based on inventory levels

© 2026 Cisco and/or its affiliates. All rights reserved.

Let's breakdown the dynamic fill

```
<svg xmlns="http://www.w3.org/2000/svg" width="326" height="415" viewBox="0 0 326 415" fill="none">
  <defs>
    <linearGradient x1="0" x2="0" y1="1" y2="0" id="fill_tablet" fill="#FFF">
      <stop offset="0%" stop-opacity="1" stop-color="#7B56DB"/>
      <stop offset="$Inventory totals:result.current_tablets$%" stop-opacity="1" stop-color="#7B56DB"/>
      <stop offset="$Inventory totals:result.current_tablets$%" stop-opacity="1" stop-color="#FFF"/>
      <stop offset="100%" stop-opacity="1" stop-color="#FFF"/>
    </linearGradient>
  </defs>
  <path id="tablet" stroke-width="8.2029" fill="url(#fill_tablet)" stroke="#c3cbd4" d="M93.7386 64.891C93.5076 64.891 81.8406 67.9618 80.6954 68.
  2763C76.7167 69.6896 73.3049 73.5571 71.6882 78.6381L28.961 212.709C27.4598 217.79 27.6907 223.333 29.5384 228.414C31.2706 233.496 34.6195 237.
  884 39.0077 240.54L218.462 348.628C222.735 351.284 226.861 351.315 231.6 350.268C231.6 350.268 241.211 348.313 245.484 346.581C249.295 344.849
  252.528 341.269 254.145 336.188L296.872 203.156C300.336 192.647 295.717 179.714 286.825 174.286L107.371 66.1976C102.752 63.3107 97.9018 63.932
  93.7386 64.891Z"/>
</svg>
```

3
4

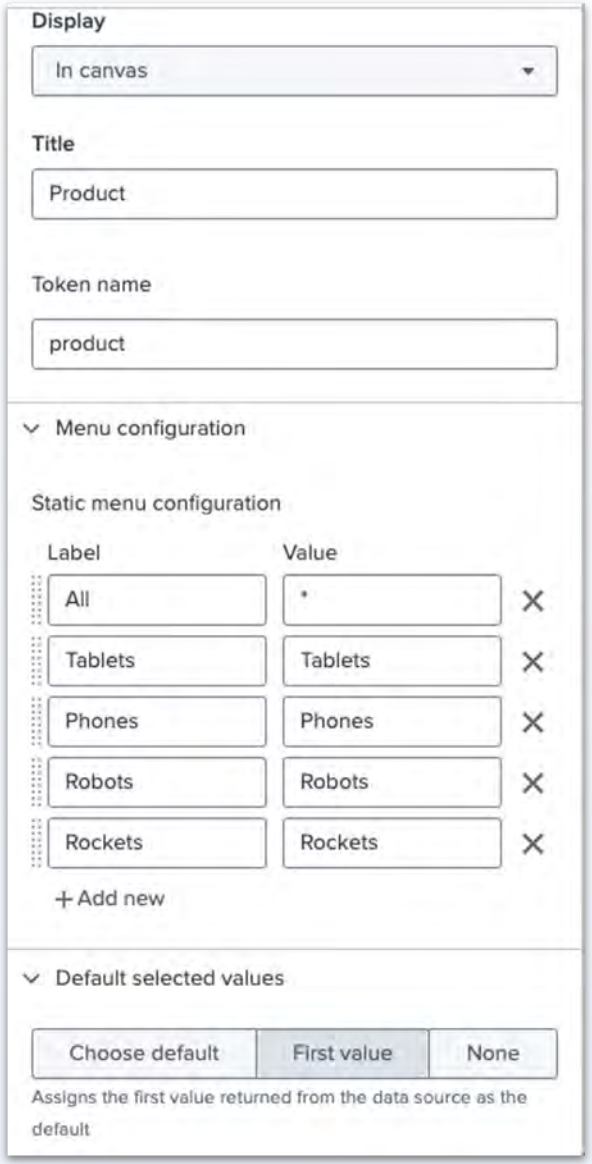
3. Each <stop> element is where we define positions and colors
    a. For example starting at 0% up to inventory levels, we want purple (#7B56DB"). Above the inventory level, we want white (#FFF)
4. This is where we are embedding the search results from our inventory search to determine how full the SVG should be

© 2026 Cisco and/or its affiliates. All rights reserved.

Exercise 10: Configure additional interactions

Filter the map by product

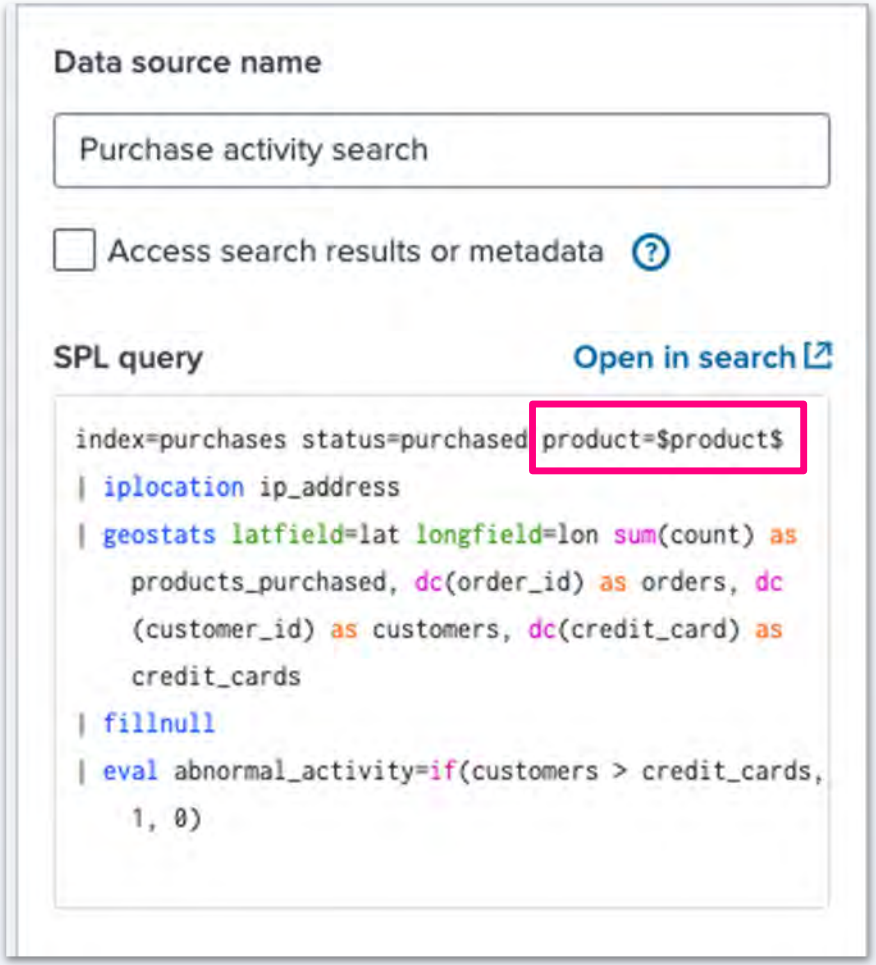
1. Add a dropdown input
2. Under **Display,** select **In canvas**
3. Reposition and resize the dropdown to be next to the **Purchase activity** custom map label
    a. Title: Product
    b. Token name: product
    c. Menu items - Label (value)
        i. All (*)
        ii. Tablets
        iii. Phones
        iv. Robots
        v. Rockets
    d. Default selected values: First value



© 2026 Cisco and/or its affiliates. All rights reserved.

Filter the map by product

4. Select the map
5. Edit the **Purchase activity search**
6. Add product=$product$ to the first pipe
7. Select **Apply and close**
8. Select different products from the dropdown to see the map update



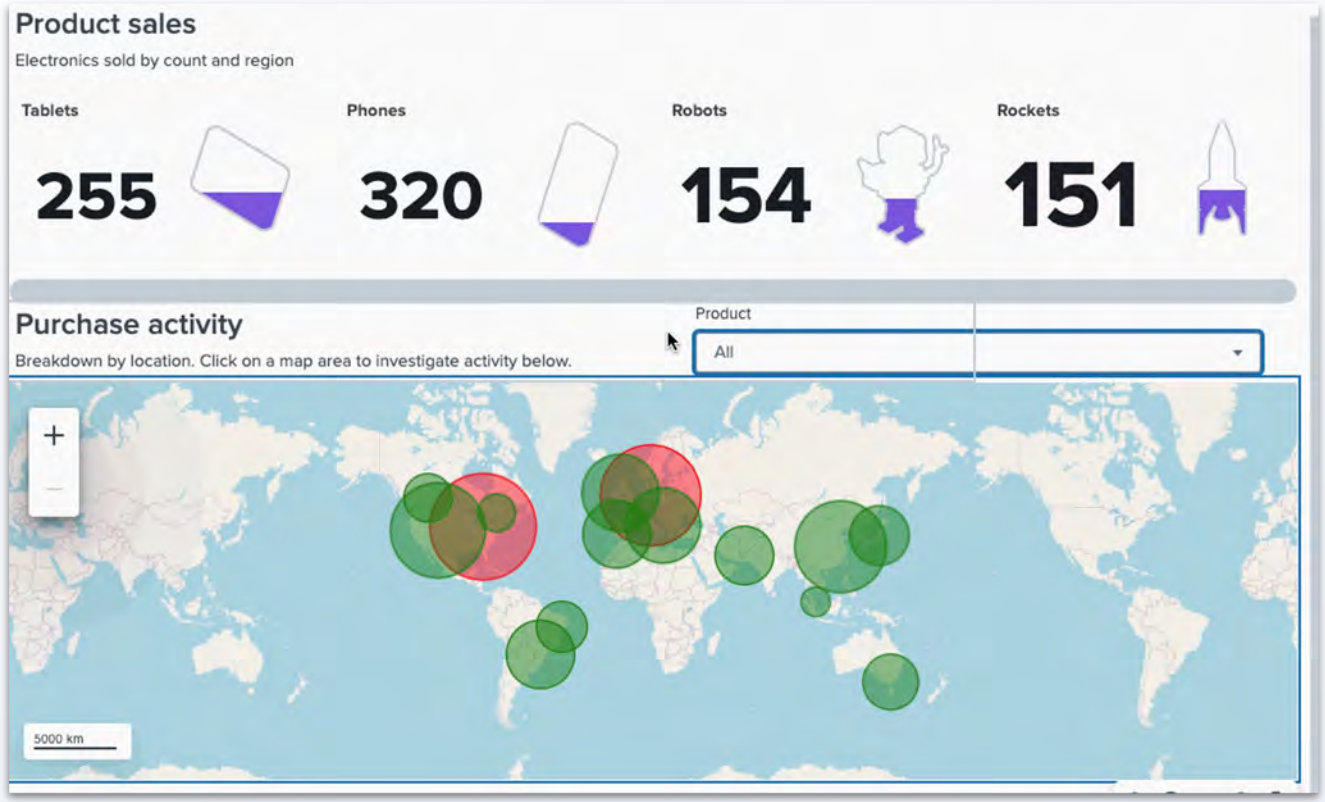
© 2026 Cisco and/or its affiliates. All rights reserved.

Configure interactions to set the product filter

Multiple interactions can set a token value

9. Select the **Tablets** single value
10. Under **Interactions**, select **+ Add interaction**
11. Under **On click**, select **Set tokens**
    a. From the dropdown, change from Use Predefined Token to Enter Static Value
    b. Token name: product
    c. Token value: name
12. Select **Apply**
13. Repeat for Phones, Robots, and Rockets
14. Save your dashboard and go to View mode

Select one of the product KPIs, and notice the Product input and map update accordingly



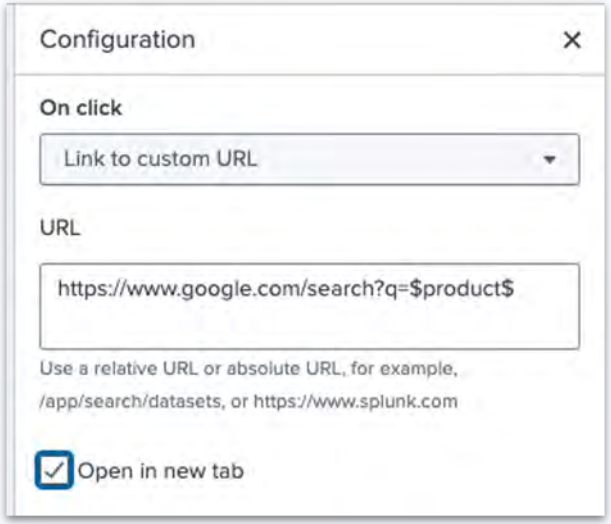
© 2026 Cisco and/or its affiliates. All rights reserved.

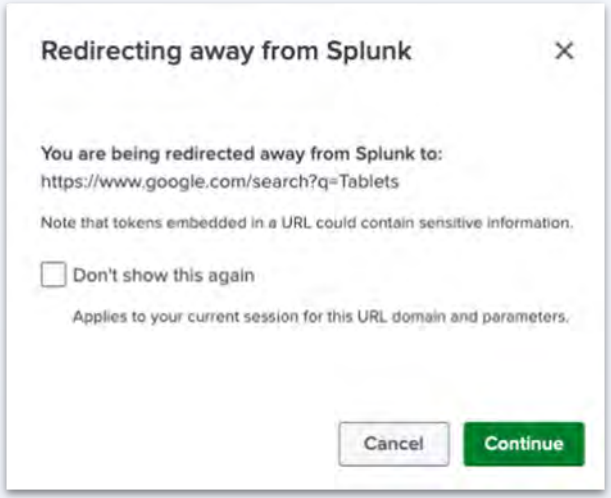
Set up additional interactions

You can configure multiple interactions on a given visualization. The order in which the interactions are listed are the order in which they will execute.

1. Back in Edit mode, select the **Tablets** KPIs
2. Under **Interactions**, select **+ Add interaction**
3. Under **On click**, select **Link to custom URL**
   a. https://www.google.com/search?q=$product$
   b. Check the box to Open in new tab
4. Select **Apply**

Save and go to View mode. Select the Tablet and see that the Product input is set to Tablet **and** you are prompted to navigate to Google.

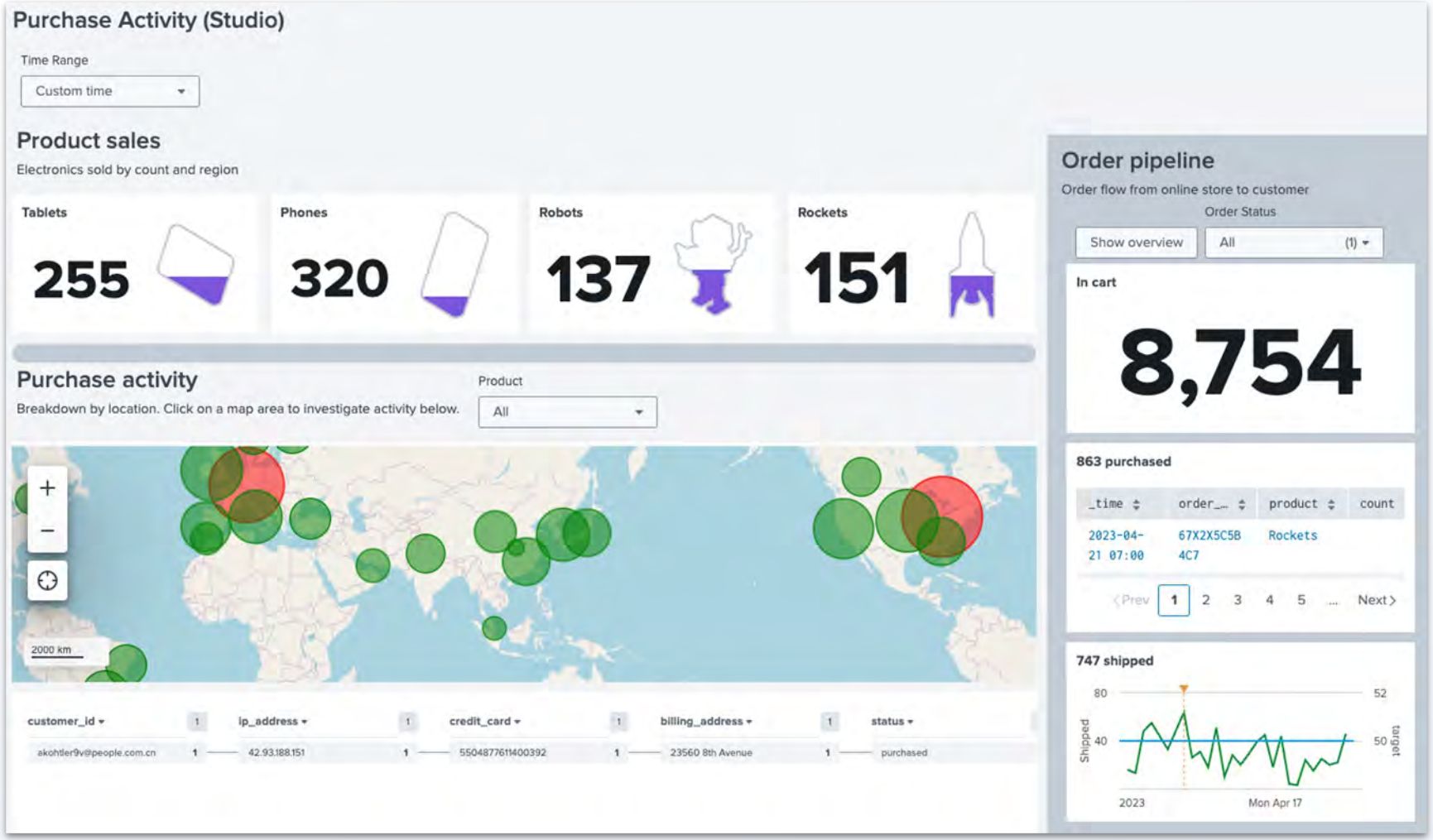




© 2026 Cisco and/or its affiliates. All rights reserved.

Your dashboard should now look like this





© 2026 Cisco and/or its affiliates. All rights reserved.

Design best practices for your next Studio dashboard

Design tip 1:
Use viewer-centered dashboard design

1. **What decisions will your dashboard likely be informing?**
   - Will it be important for your viewers to compare one value to another, see a change over time, or drill down on a certain data point?
   - Design your dashboard so that viewers can easily make these key observations
   - For example, use a bar chart to compare values, and a line chart to see changes over time
2. **Leverage consistency and familiarity when possible**
   - Familiarity can help viewers orient themselves, and quickly understand the dashboard
   - For example, if your organization uses a certain color palette to communicate certain warning levels, try to stay consistent with the existing palette.

Check out more design best practices here: https://www.splunk.com/en_us/form/dashboard-design-best-practices.html

© 2026 Cisco and/or its affiliates. All rights reserved.

Design tip 2:
Communication best practices

1. **Consider information hierarchy.**
   - Place high-level information at the top of the dashboard.
   - Add charts that provide details about the high-level information below.
2. **When necessary, use text to instruct users on the functionality of your dashboard.**
   - For example, using Markdown to instruct users to select a customer from the table.
3. **Use short, descriptive titles for charts.**
4. **Be intentional about chart type.**
   - Dashboard Design: Visualization Choices and Configurations

© 2026 Cisco and/or its affiliates. All rights reserved.

Revisiting our learning objectives

Today you learned how to:

- ✓ Configure interactivity
- ✓ Configure conditional show/hide
- ✓ Apply options with Dynamic Options Syntax
- ✓ Add multiple data sources to a visualization
- ✓ Add custom dynamic SVGs
- ✓ Apply best practices
  - ✓ Base and chain searches
  - ✓ Reusing searches for multiple visualizations
  - ✓ Design best practices



© 2026 Cisco and/or its affiliates. All rights reserved.

Additional resources

- Visit the in-product **Examples Hub**
- Check out the Dashboard Studio **docs**
- Read our Dashboard Studio **blog posts** for new features and tips and tricks
- Join the **#dashboard_studio** Splunk user groups Slack channel
- Take an **EDU course** about dashboards
  - Intro to Dashboards
  - Dynamic Dashboards
- Watch the dashboards performance best practices tech talk
  - Improving Dashboard Performance and Resource Usage



© 2026 Cisco and/or its affiliates. All rights reserved.

splunk> by cisco

Thank you

cisco